# 10 Datenbanken


1. Motivation
2. Datenorganisation und Datenbankkonzept
3. Semantische Datenmodellierung
4. Umsetzung in Datenbanken
5. Datenbanknutzung mit SQL
6. Transaktionsmanagement
7. Datenbankentwicklung
8. Datenbanken und IT-Sicherheit
9. Systemarchitektur
**10. NoSQL Datenbanken**
11. Verteilte Datenbanken

# 10 Lernziele

- Sie haben einen Überblick über die verschiedensten Entwicklungen im Bereich der „NoSQL“-Systeme.
- Sie verstehen, warum es so viele verschiedene „NoSQL“- Lösungen gibt.
- Sie erkennen Anwendungsfälle für verschiedene „NoSQL“-Datenbanken

# 10 Vor- und Nachteile von SQL-Datenbanken

**SQL-Ziel**:
Konsistenzerhöhung durch Redundanzvermeidung
(Normalisierung)

- ✚ Logischer Aufbau
- ✚ Strikte Regeln und hoher Grad an Normalisierung
- ✚ ACID Eigenschaften (Transaktionssicherheit)
- ✚ Standardisierte APIs
- ▬ Mehr Leistung (Skalierbarkeit) durch schnellere Rechner ist sehr teuer
- ▬ Je komplexer die Beziehungen der Daten, desto exponentiell komplexer wird das relationale DBMS

# 10 Kritik an SQL-Datenbanken

- SQL ist nicht skalierbar
  - Normalisierung von Relationen verlangt viele Verknüpfungen
  - Viele Integritätsbedingungen zu prüfen

  ➔ Langsam!

- Starre SQL-Tabellen sind nicht flexibel genug
  - Schwach typisierte Tabellen
    (Tupel weichen in den tatsächlich genutzten Attributen ab)
    - Viele Nullwerte wenn alle potentiellen Attribute definiert sind
    - Alternativ Aufspaltung auf viele Tabellen

  ➔ Langsam und platz-verschwenderisch.

# 10 Warum NoSQL?

- Drastisch gestiegene Datenmengen
  - Skalierbarkeit
  - Lese-/Schreibperformance
- Viel größere Systeme und somit:
  - Großer Datenbestand
  - Sehr häufiges Lesen
  - Parallele Zugriffe
- Viele Anwendungsfälle mit weniger strukturierten Daten
- Spezielle Daten / Abfragen

Verschiedene Anwendungsfälle haben
verschiedene Anforderungen

# 10 Was bedeutet NoSQL?

- Sammelbegriff verschiedener Datenbanktechnologien
- „Kein SQL“ (ursprünglich)
- Datenbanken, die kein relationales Datenmodell verwenden.
- „Gegen das System!“

- „Not only SQL“ (heute)
  - Relationale Datenbanken werden nicht verschwinden

# 10 Zur Erinnerung: Datenbankmodell / -logik

- Abstraktion von Bits und Bytes
- Hilft Daten strukturiert zu betrachten
- Hat Auswirkungen auf die Realisierung

**Beispiele für Daten:**

- Relationenmodell
- Dokumente (JSON, XML, etc.)
- Schlüssel-Wert-Paar (Map)
- Graphen

# 10 Datenschemata in NoSQL

- NoSQL-Datenbanken sind im Vergleich zu relationalen Datenbanken durch **Schemalosigkeit/-freiheit** gekennzeichnet
- Ein **indirektes Schema** wird jedoch im Applikationscode definiert
- Das indirekte Schema orientiert sich an
  - den **Anforderungen**, die die Applikation zu erfüllen hat und
  - den **Limitierungen/Fähigkeiten** der verwendeten NoSQL-Datenbank
- Einige Implementierungen von NoSQL-Datenbanken bieten die Möglichkeit **Validierungsregeln** für Datenmutationen (Ändern, Einfügen, Löschen) zu definieren

# 10 “Routenplanung”

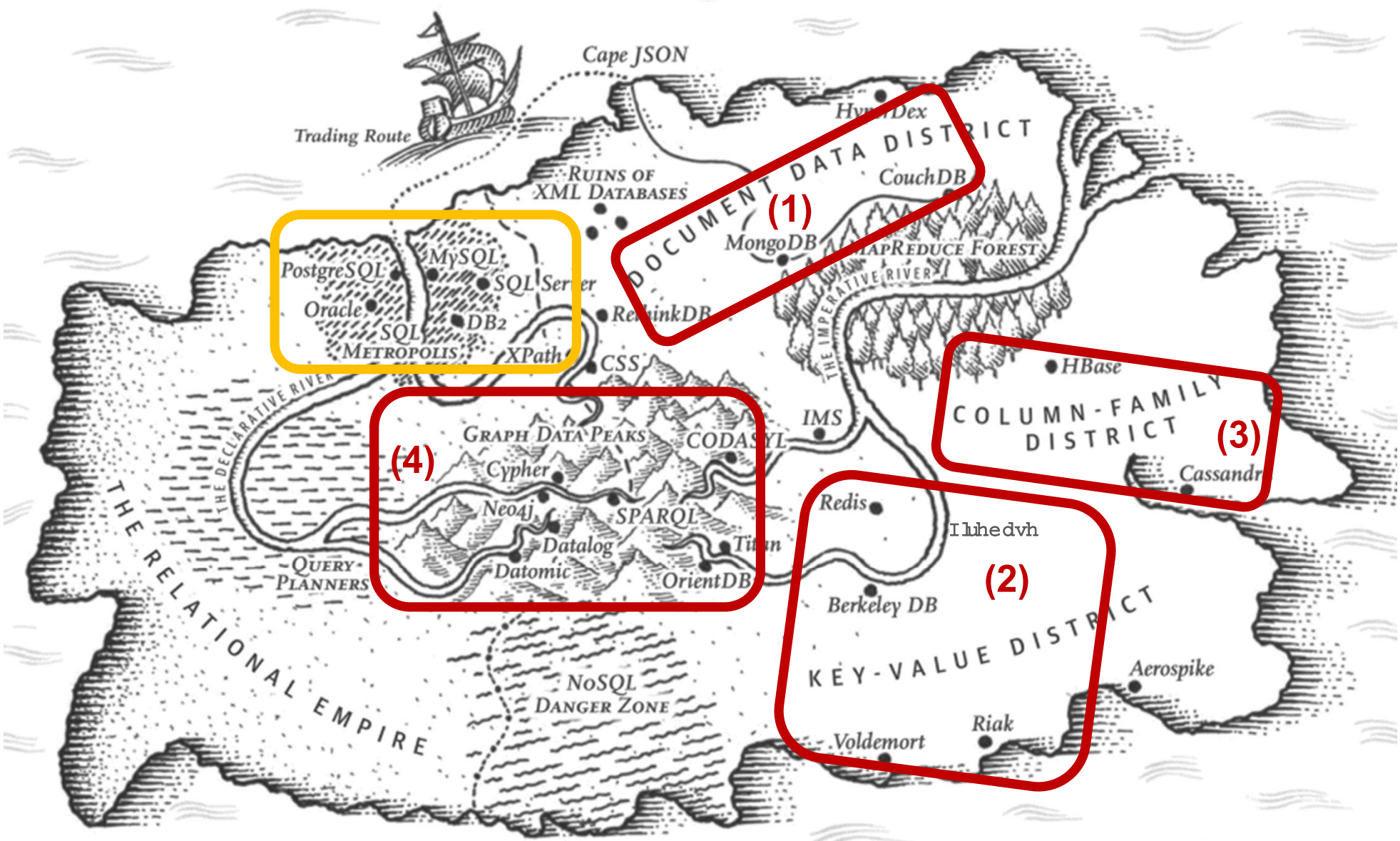
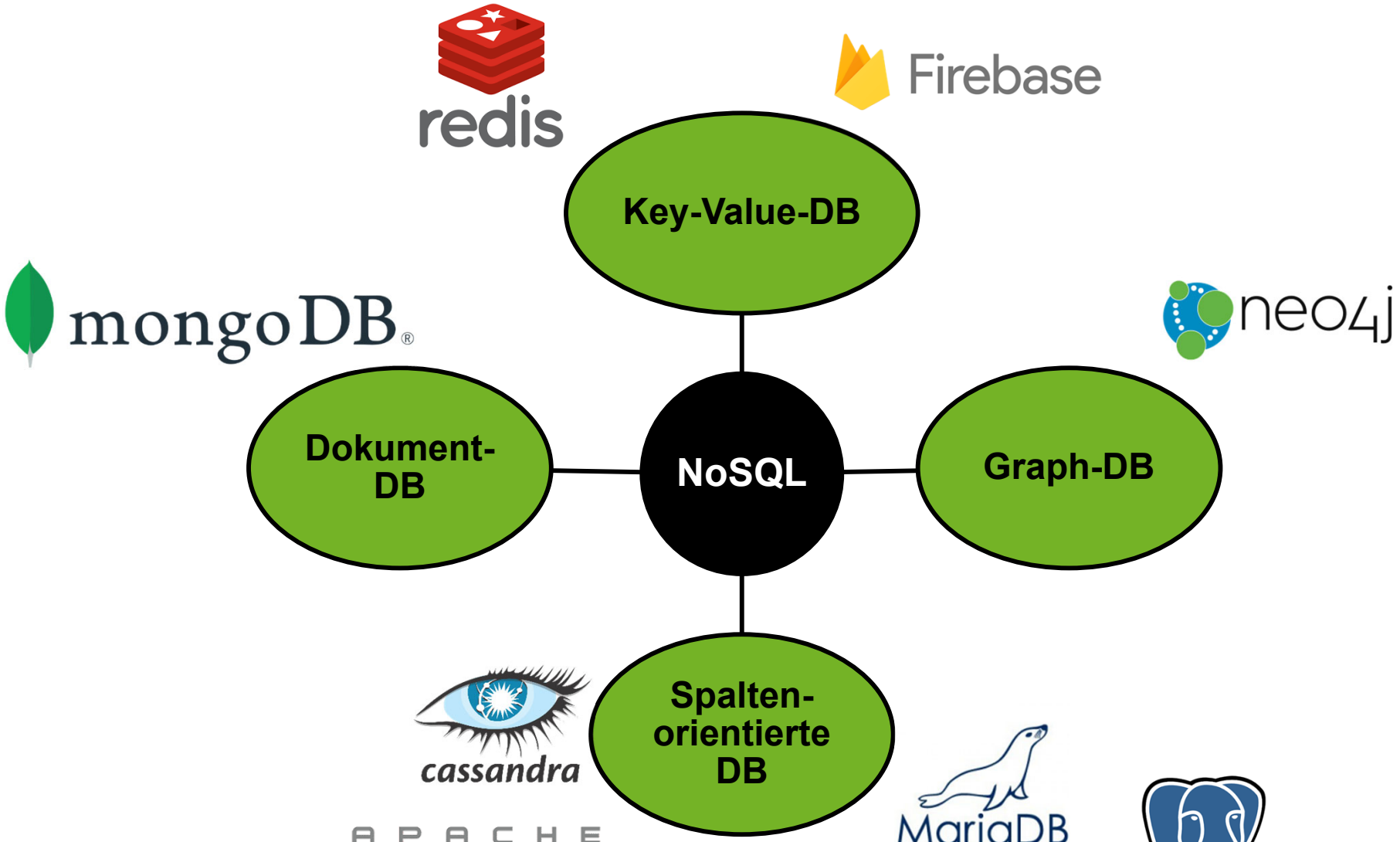



Bild: http://muratbuffalo.blogspot.com/2024/08/designing-data-intensive-applications.html

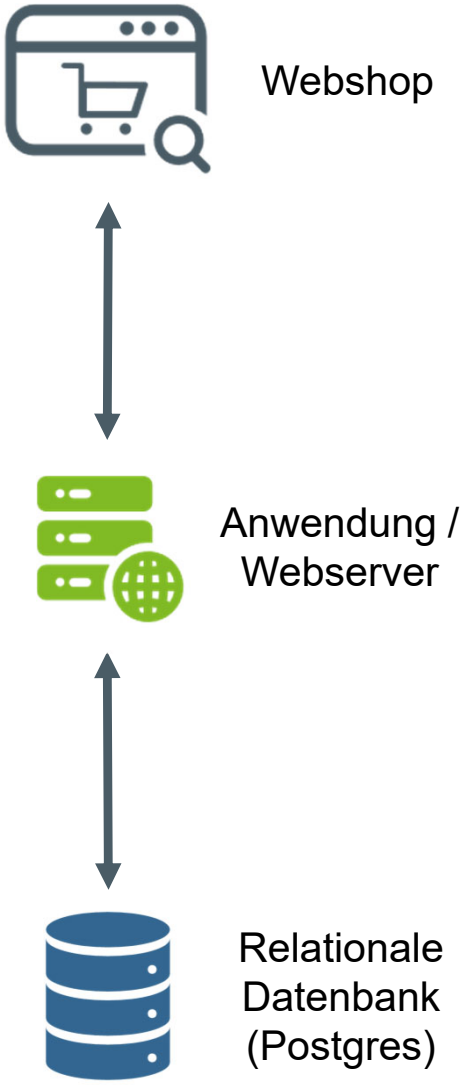
# 10 Kerntechnologien im Bereich NoSQL



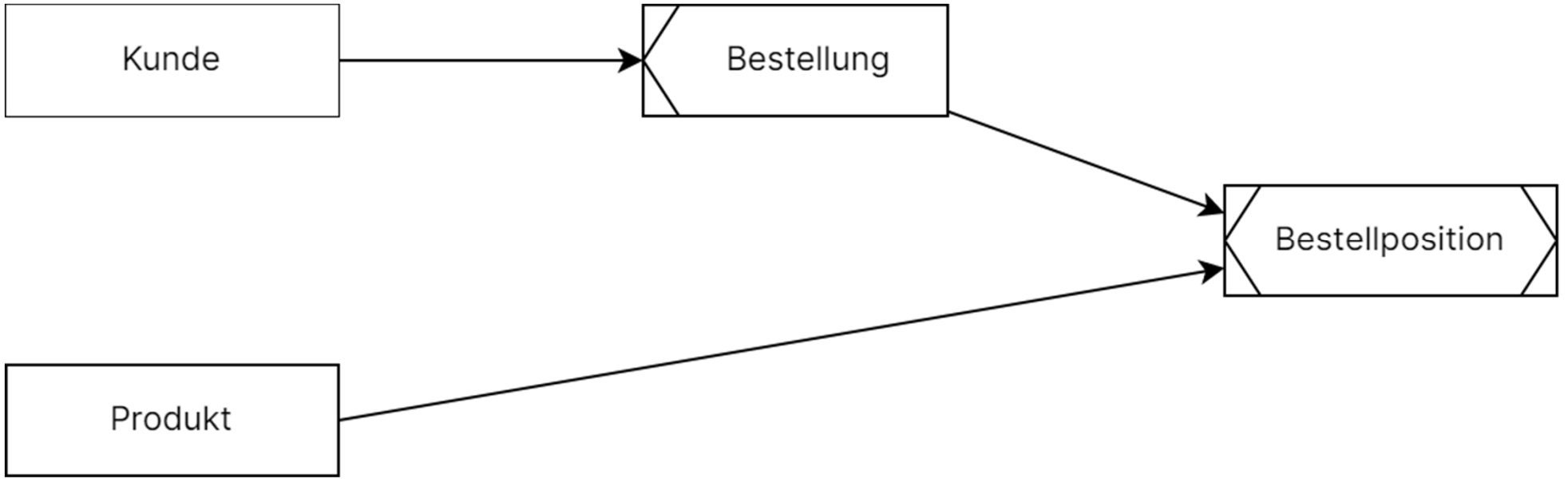
# Fallbeispiel für SQL- und NoSQL-Datenbanken: Onlineshop

# 10 Der Onlineshop

- Verkauft Bücher
- Neue Artikel auf Startseite
- Daten:
  - Kunden
  - Bestellungen
  - Produkte



# 10 Der Onlineshop (SERM)



# 10 Datenschema SQL

```
create table kunde (
    kundennummer serial
        primary key,
    name varchar(50),
    vorname varchar(50),
    strasse varchar(70),
    hausnummer varchar(6),
    plz varchar(10),
    stadt varchar(50)
);
```

```
create table produkt (
    produktnummer serial
        primary key,
    bezeichnung varchar(30),
    beschreibung text,
    isbn varchar(15),
    preis money,
    autor varchar(80),
    verlag varchar(80)
);
```

# 10 Datenschema SQL

```
create table bestellung (
    bestellnummer serial
        primary key,
    kundennummer integer
        references
        kunde(kundennummer),
    zeitstempel timestamp
);
```

```
create table bestellposition (
  bestellnummer integer
      references
      bestellung(bestellnummer),
  produktnummer integer
      references
      produkt(produktnummer),
  anzahl smallint,
  primary key (bestellnummer,
      produktnummer)
);
```

Aufgabe 1: *Es soll ein neuer Kunde (Kundennummer: 73234) mit dem Namen Hans Müller und der Email-Adresse hans.mueller@mni.thm.de angelegt werden. Danach soll anhand der Kundennummer der Name ausgegeben werden.*

SQL:

```
INSERT INTO Kunde (KNr, Name, Vorname, Email)
VALUES (73234, `Müller`, `Hans`,`hans.mueller@mni.thm.de`);

SELECT Name FROM Kunde WHERE KNr=73234;

>>> Müller
```

Aufgabe 2: *Zu einem Kunden ist nur die Email-Adresse bekannt.*
*Wie gelangt man an seine Kundennummer?*

SQL:

```
SELECT KNr FROM Kunde WHERE email='hans.mueller@mni.thm.de';
```

```
>>> 73234
```

Aufgabe 3: *Für die Produkte wird in der Warenkorbansicht des Online-Shops eine Listendarstellung gewählt, wo nur wenige Produktinformationen (PNr, Bild, Bezeichnung und Preis) angezeigt werden sollen.*

SQL:

```
SELECT PNr, BildUrl, Bezeichnung, Preis FROM Produkt

>>> '1234','X:\web\pics\1\192.jpg','New Opportunities for
Connected Data','20,99'
...
```

# 10 Merkmale

- Strukturierte Daten
  - Produkte sind Bücher → ISBN, Festpreis
  - Titel
  - Klappentext
  - Keine Sonderattribute
- 7000 Bestellungen am Tag, 50000 Kunden
  → Problemlos händelbar. *
- Erweiterbarkeit?

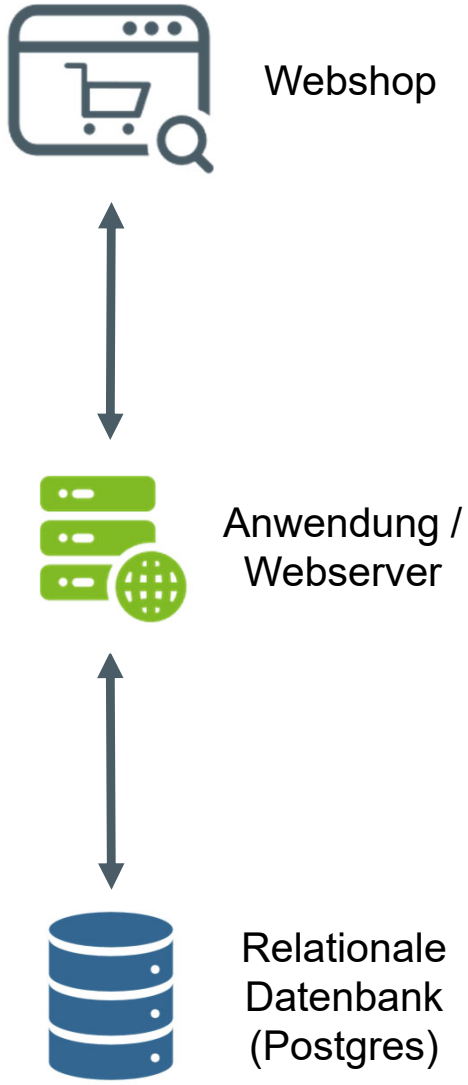
* Setzt sinnvolle Indizes und optimierte Abfragen voraus.

# 10 Der Onlineshop

- Neue Artikel auf Startseite
- Daten:
  - Kunden
  - Bestellungen
  - Produkte

Neue Anforderung:

- Verkauft verschiedenste Produkte



# 10 Neue Anforderung

- Neue Artikel: Prozessoren
  - EAN
  - Hersteller
  - Taktfrequenz
  - Anzahl Kerne
  - L3-Cache
  - L2-Cache
  - L1-Cache
  - Stromverbrauch
  - Sockel
  - …

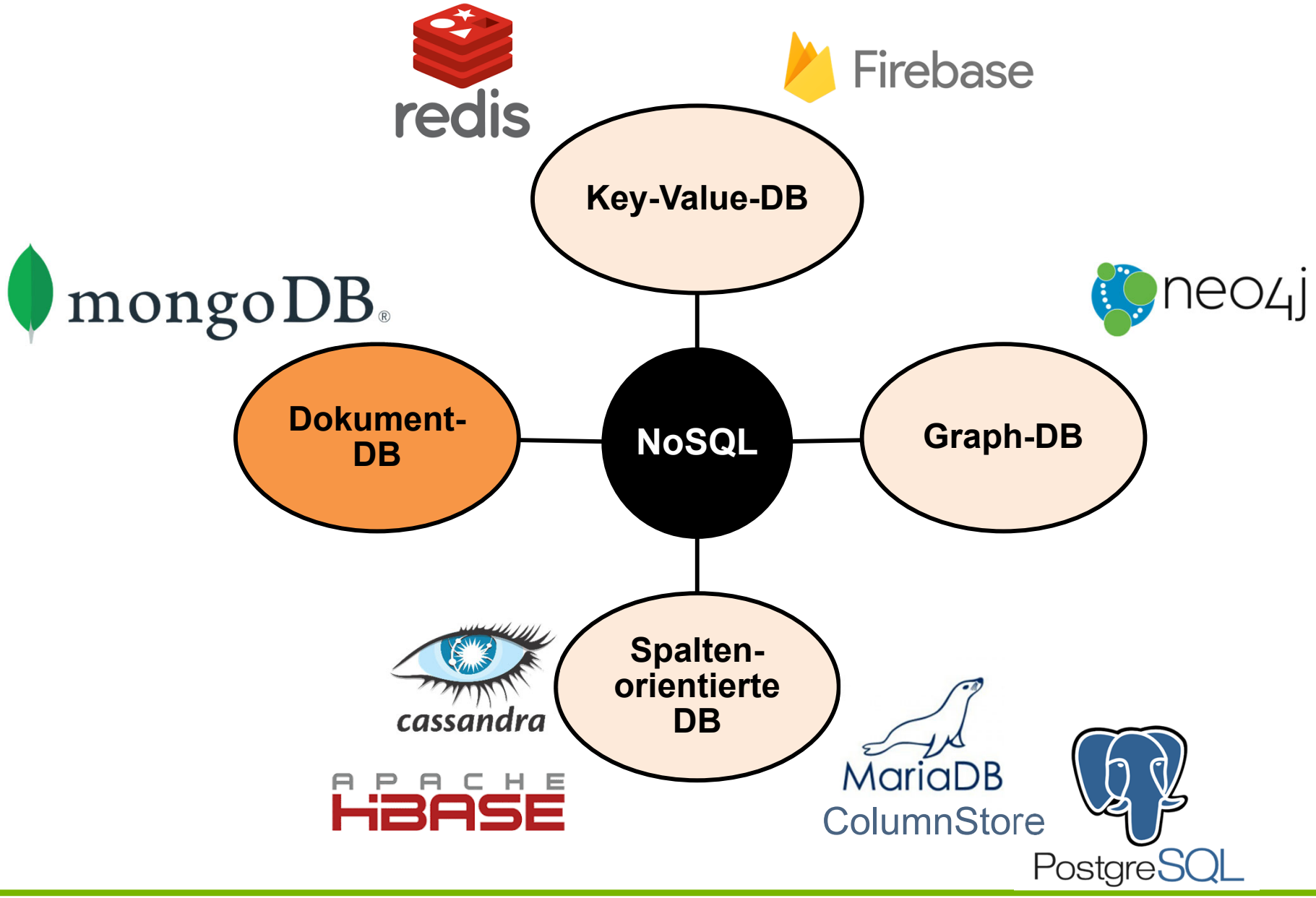
```
alter table produkt
add column kategorie references
    kategorie(kategorienummer),
add column ean varchar(13),
add column hersteller varchar(50),
add column taktfrequenz varchar(10),
add column anz_kerne smallint,
add column l3_cache varchar(15),
add column l2_cache varchar(15),
add column l1_cache varchar(15),
add column stromverbrauch integer,
add column sockel varchar(10);
```

# 10 Ergebnis

| PNr | Bezeichnung | ISBN | EAN | Verlag | Kategorie | Taktfrequenz | Kerne |
|---|---|---|---|---|---|---|---|
| 1 | StackOverflow | 123-4-56-789012-4 | *NULL* | O'RLY? | Bücher | *NULL* | *NULL* |
| 2 | Intel i42 | *NULL* | 7744233123398 | *NULL* | CPU | 3000 MHz | 8 |
| 3 | Datenbanken | 987-6-54-321098-7 | *NULL* | Springer | Bücher | *NULL* | *NULL* |

- Viele Nullwerte bei speziellen Attributen
- Alternative?
  - Viele Tabellen
- Konsequenz?
  - Umständliche / Mehrere Joins

# 10 Kerntechnologien im Bereich NoSQL



# 10 Dokumentenorientierte Datenbanken

- Daten werden als Dokument gespeichert.
  - Z.B.: JSON, XML, BSON, YAML
- Dokumente haben eine Baumstruktur
  - One-to-Many Beziehungen intuitiv darstellbar
- Besonders geeignet für in sich geschlossene Daten
  - Artikeldaten mit Zusätzen
  - Lebensläufe
  - Kochrezepte
- Bekannte dokumentenorientierte Datenbanken:
  - MongoDB, CouchDB, RavenDB









# 10 Produkte

```
{
    "Produktnummer": 1,
    "Bezeichnung": "Stack Overflow
      für Dummies",
    "Kategorie": "Bücher",
    "ISBN": "123-4-56-789012-4",
    "Verlag": "O'RLY?",
    "Klappentext": "Stack Overflow
      ist die wichtigste Ressource
      für Informatiker..."
}
```

```
{
    "Produktnummer": 2,
    "Bezeichnung": "Intel i42",
    "Kategorie": "CPU",
    "EAN": "7744233123398",
    "Taktfrequenz": "3000 MHz",
    "Kerne": 8,
    "Stromverbrauch": 120,
    „RAM-Takt": [2666, 2833, 3000]
}
```

# 10 Bestellungen

```
{
    "Bestellnummer": 41233,
    "Kundennummer": 42,
    "Zeitstempel": "17.05.2022 17:06:51+01",
    "Positionen": [{
            "Produktnummer": 57,
            "Anzahl": 5
        }, {
            "Produktnummer": 751231541,
            "Anzahl": 1
        }]
}
```

Kann es hier Probleme geben?

# 10 Eigenschaften dokumentenorientierter Datenb.

- Schemalosigkeit / Flexibles Schema
  - Dokumente müssen nicht die gleiche Struktur haben.
  - Keine Fremdschlüssel
  - Schema-on-read vs. Schema-on-write (!)
  - Dokumente können sich einfach ändern
- Hohe Skalierbarkeit, ggf. Dokumente auf versch. Servern
- Schnellere Abfragen
- Besonders geeignet für wenig komplexe Beziehungen

# Grundlagen NoSQL: Dokument-Datenbanken

- Zu jedem Dokument werden durch die Datenbank **Metadaten** gespeichert
  - Hierdurch werden **komplexere Queries** ähnlich zu den SQL-Abfragen möglich
  - Es können auch **einzelne Attribute eines Objektes** zurückgegeben werden
- Ähnliche Dokumente werden zu **Collections** zusammengefasst:

| SQL-Datenbanken | Dokumentorientierte NoSQL-Datenbanken |
|---|---|
| Table | Collection |
| Row / Record | Document |
| Column / Attribut | Field (bzw. Key) |

# 10 JSON-Beispiel



# Datenschema NoSQL: Dokument-Datenbanken




- Für einzelne Collections können in einigen dokumentorientierten NoSQL-Datenbanken Validierungsregeln für Datenmutationen (Ändern, Einfügen, Löschen) definiert werden

- Beispiel MonogoDB:

```
db.createCollection( "Kunde", { validator: { $or:
    [
      { PLZ: { $type: "string" } },
      { Email: { $type: "string" } },
      { Land: { $in: ["DE", "AT", "CH"] } }
    ]
  }
 }
)
```

# Datenmanipulation
# NoSQL: Dokument-Datenbanken (mongoDB)

Aufgabe 1: *Es soll ein neuer Kunde (Kundennummer: 73234) mit dem Namen Hans Müller und der Email-Adresse hans.mueller@mni.thm.de angelegt werden. Danach soll anhand der Kundennummer der Name ausgegeben werden.*

SQL:

```
INSERT INTO Kunde (KNr, Name, Vorname, Email)
VALUES (`73234`,`Müller`,`Hans`,`hans.mueller
@mni.thm.de`);
```

Kunden in mongoDB speichern:

```
db.Kunde.insert( {
    _id: "73234", name: "Müller", vorname: "Hans",
    email: "hans.mueller@mni.thm.de" } )

db.Kunde.find()
>>> { "_id" : "73234", "name" : "Müller", "vorname" :
"Hans", "email" : "hans.mueller@mni.thm.de" }

db.Kunde.find( { _id: "73234" }, { name: 1, _id: 0 } )
>>> { "name" : "Müller" }
```

1: Das Attribut soll ausgegeben werden
0: Das Attribut soll nicht ausgegeben werden

# Datenmanipulation
# NoSQL: Dokument-Datenbanken (mongoDB)

Aufgabe 2: *Zu einem Kunden ist nur die Email-Adresse bekannt.*
*Wie gelangt man an seine Kundennummer?*

SQL:

```
SELECT KNr FROM Kunde WHERE
email='hans.mueller@mni.thm.de';
```

mongoDB:

Bedingungen sind nicht nur an den Schlüssel, sondern an Attribute möglich.

```
db.Kunde.find( { email: "hans.mueller@mni.thm.de" },
               { _id: 1 } )
>>> { "_id" : ObjectId("73234") }
```

---

**Weiteres Beispiel:**

```
db.Kunde.find( { vorname: "Hans",
                 name:{ $in:['Müller','Maier']}
               } )
```

Alle Kunden mit Vornamen Hans und Nachnamen Müller oder Maier

# Datenmanipulation
# NoSQL: Dokument-Datenbanken (mongoDB)

Aufgabe 3: *Aufträge und Produkte speichern?*

```
db.orders.insert( [
 { "_id" : 1, "item" : "kiwi", "price" : 12, "quan" : 8 },
 { "_id" : 2, "item" : "pecans", "price" : 20, "quan" : 1 },
 { "_id" : 3  }
] )

db.inventory.insert( [
 { "_id" : 1, "sku" : "kiwi", "descr": "...", "instock" : 120 },
 { "_id" : 2, "sku" : "bread", "descr": "...", "instock" : 80 },
 { "_id" : 3, "sku" : "cashews", "descr": "...", "instock" : 60 },
 { "_id" : 4, "sku" : "pecans", "descr": "...", "instock" : 70 },
 { "_id" : 5, "sku": null, "descr": "Incomplete" },
 { "_id" : 6 }
] )
```

# Datenmanipulation
# NoSQL: Dokument-Datenbanken (mongoDB)

Aufgabe 4: *Zu Aufträgen die Produktbeschreibungen ausgeben?*

SQL:

```
SELECT *
FROM orders, inventory
WHERE orders.item = inventory.sku
```

mongoDB: Eine Aggregationspipeline besteht aus einer oder mehreren Phasen, die Dokumente verarbeiten: Jede Phase führt einen Vorgang an den Eingabedokumenten aus.

```
db.orders.aggregate( [ {
      $lookup:
        {
          from: "inventory",
          localField: "item",
          foreignField: "sku",
          as: "inventory_docs"
        }
} ] )
```

Output

```
{   "_id" : 1,
    "item" : "kiwi",
    "price" : 12,
    "quantity" : 8,
    "inventory_docs" : [
      { "_id" : 1,
        "sku" : "kiwi",
        "descr" : "...",
        "instock" : 120
      }]
} …
```

Aufgabe 4: *Ein oder alle Produkte löschen?*

```
db.inventory.remove({ sku: "bread" });
>>> WriteResult({ "nRemoved" : 1 })


db.inventory.remove({});
>>> WriteResult({ "nRemoved" : 4 })
```

# 10 Anwendungsfall erkennen

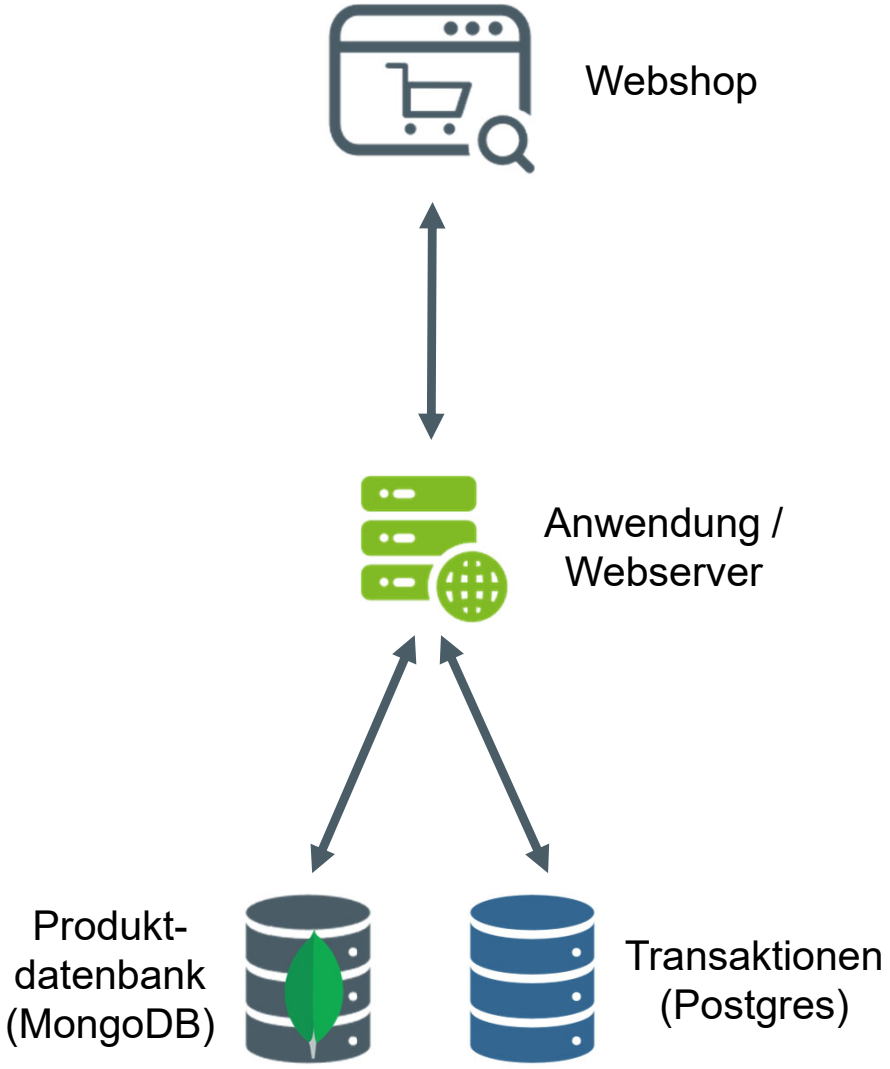
- Wann bieten sich dokumentenorientierte Datenbanken an?
  - Wenn die Daten Dokumente sind.
  - Wenn viele 1:n-Beziehungen vorliegen.
  - Wenn Datensätze sehr heterogen sind.
  - Wenn kein großer Wert auf Schema gelegt wird.
  - Sehr wenige / keine (n:m)-Beziehungen zwischen den Dokumenten (also den Datensätzen)

# 10 Der Onlineshop

- Verkauft **diverse Artikel** ✓
- Startseite:
  - Neue Artikel
- Daten:
  - Kunden
  - Bestellungen
  - Produkte

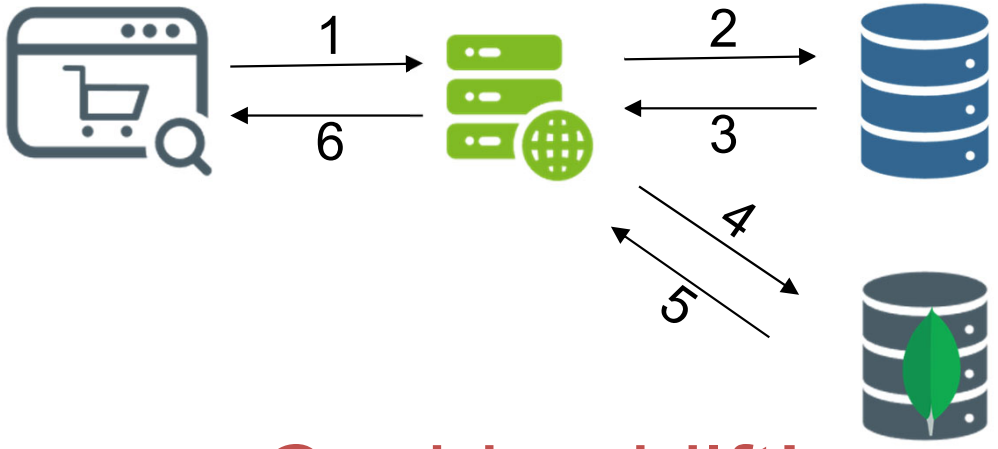
Neue Anforderung:

- Startseite mit meistverkaufte Artikel



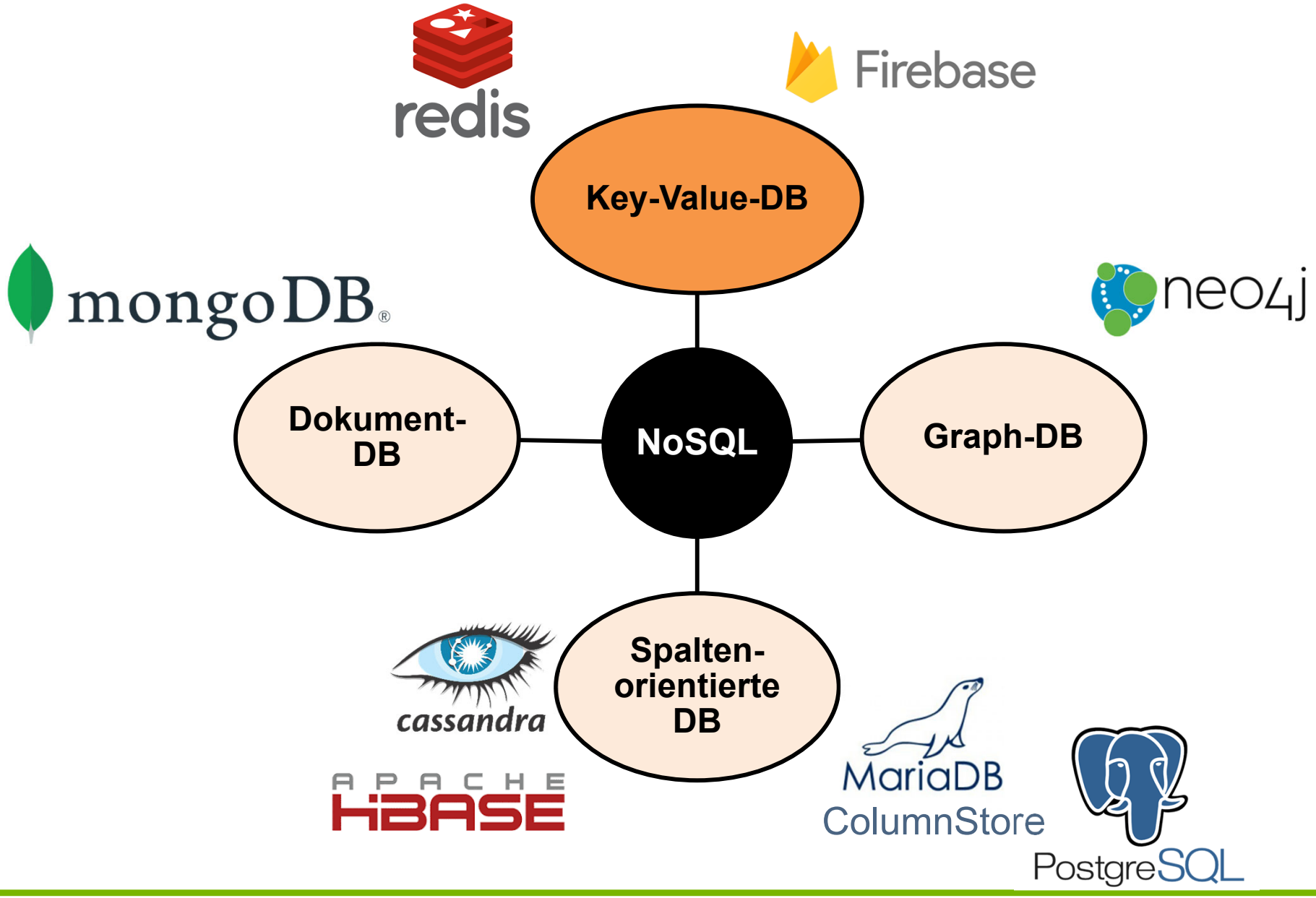
# 10 Ermitteln der meistverkauften Artikel

- Postgres:
  - Ermittle die 10 meistverkauften Artikel
- MongoDB / Dokumentdatenbank:
  - Suche die Produktdaten zu diesen 10 Artikeln




Caching hilft!

# 10 Kerntechnologien im Bereich NoSQL



# 10 Key-Value-Datenbanken







- Simpelste Form der NoSQL-Datenbanken
  - Dokumentenorientierte DBs sind im Kern auch Key-Value-DBs
- Speichern von Daten in Form von **Key-Value**-Paaren
- Sehr sehr schnell !!
- Die Datenbank „versteht“ den Inhalt nicht, der sich hinter einem Key verbirgt, keine Verknüpfungen, auch keine 1-n-Verknüpfungen
- Jeder Key fungiert als **Primary Key**
- Es existieren **keine Foreign Keys**: Konsistenzsicherung muss anders erfolgen (wenn nötig)
- **Häufige Anwendungsfälle: Caching & Authentifikation**
- Basis vieler anderer Datenbanken, z.B. dokumentenorientierte
- Anleitung zum Firebase Database ausprobieren: https://firebase.google.com/docs/firestore/quickstart?hl=de

# Key-Value-Datenbanken







- Nur einfachste Operationen
  - SET
  - GET
- Kein Schema
  - Struktur kann durch Schlüssel gegeben werden
  - Einfachste Skalierbarkeit
- Sehr eingeschränkte Abfragen

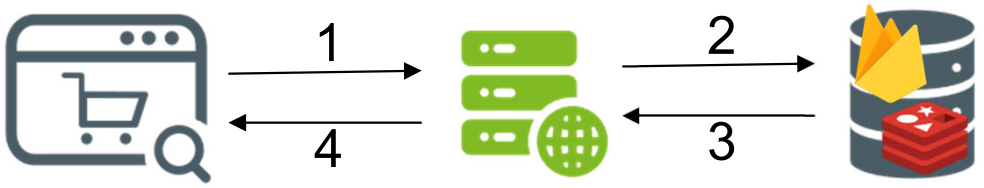
```
SET Kunde:K312352:vorname Max
SET Kunde:K312352:nachname Mustermann
SET Kunde:K312352:email max.mustermann@gmail.com
```

Schlüssel

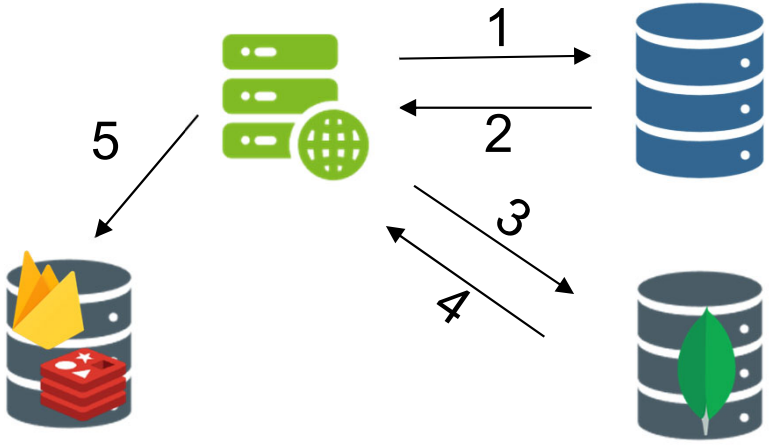
Wert

# 10 Ermitteln der meistverkauften Artikel

- Nutzeranfrage:
  - Frag den Cache nach den meistverkauften Artikeln+Beschreibung

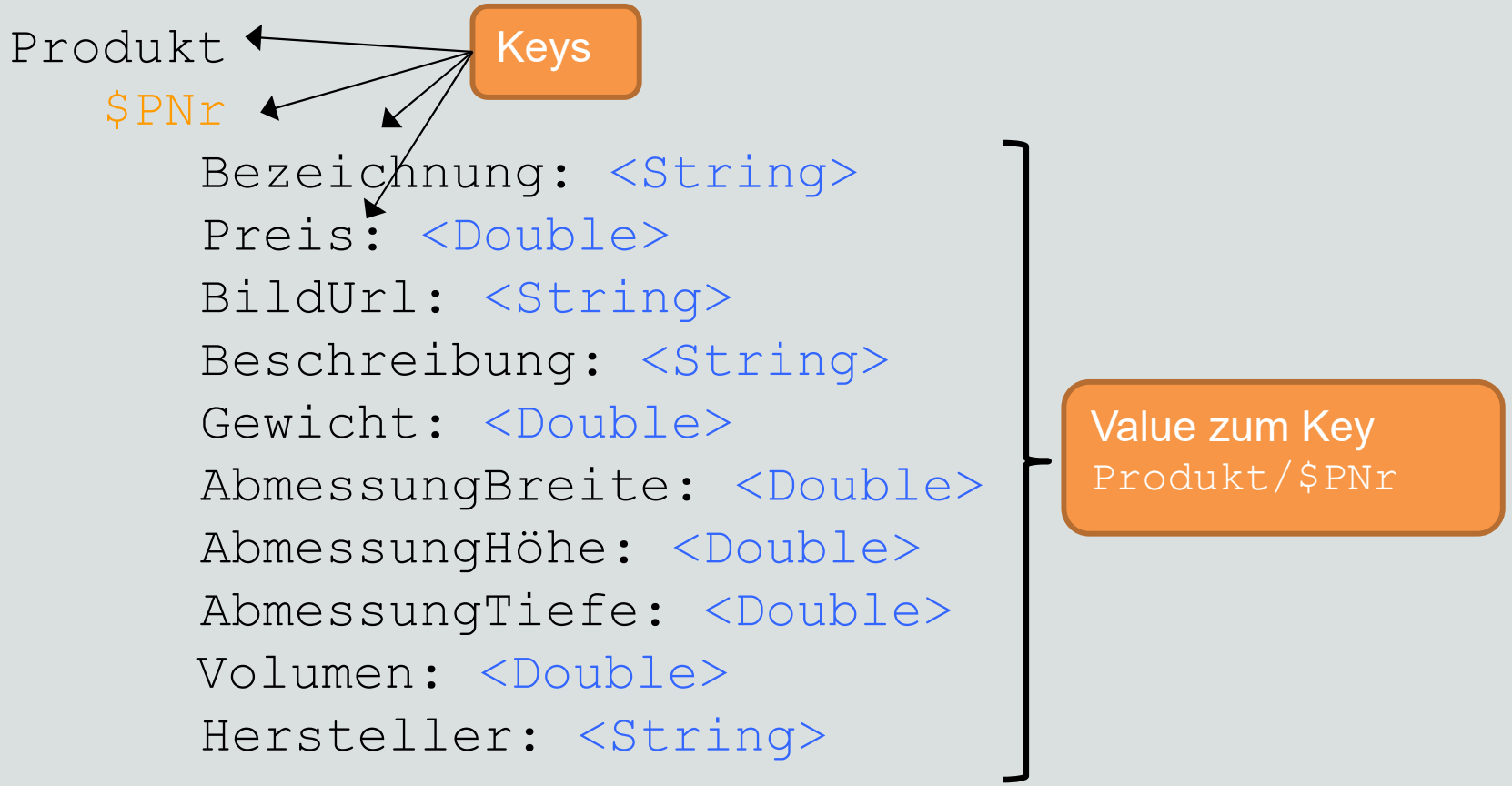



- Regelmäßig:
  - Frage die meistverkauften Artikel ab
  - Schreibe das Ergebnis in Cache

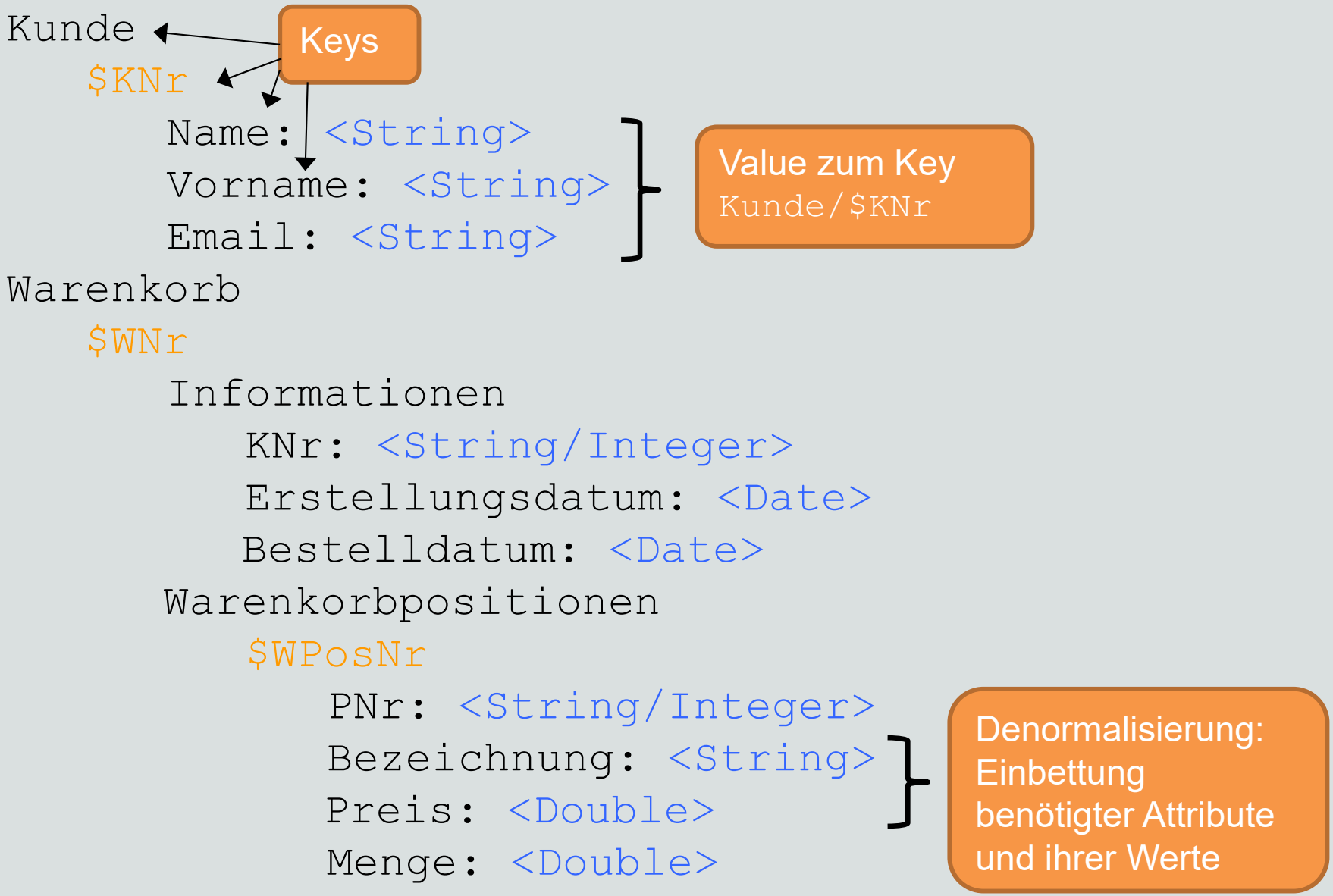



**Weitere Anwendung für Key-Value-DB?**
**Authentifikation!**

# 10 Datenschema NoSQL: Key-Value-Datenbanken (Firebase)



# Datenschema NoSQL: Key-Value-Datenbanken (Firebase)



# Datenschema NoSQL: Key-Value-Datenbanken (Firebase)

Aufgabe 1: *Es soll ein neuer Kunde (Kundennummer: 73234) mit dem Namen Hans Müller und der Email-Adresse hans.mueller@mni.thm.de angelegt werden. Danach soll anhand der Kundennummer der Name ausgegeben werden.*

SQL:

```
INSERT INTO Kunde (KNR, Name, Vorname, Email)
VALUES (`73234`,`Müller`,`Hans`,`hans.mueller
@mni.thm.de`);
```

Firebase:

```
firebase.database().ref('Kunde/73234').set({
    Name: "Müller",
    Vorname: "Hans",
    Email: "hans.mueller@mni.thm.de"
});
```

# Datenschema NoSQL: Key-Value-Datenbanken (Firebase)

Aufgabe 1: *Es soll ein neuer Kunde (Kundennummer: 73234) mit dem Namen Hans Müller und der Email-Adresse hans.mueller@mni.thm.de angelegt werden. Danach soll anhand der Kundennummer der Name ausgegeben werden.*

SQL:

```
SELECT Name FROM Kunde WHERE KNr=73234;
```

Firebase:

```
var kref = firebase.database().ref('Kunde/73234/Name');

kref.once('value', function(snapshot) {
  // der zurückgegebene snapshot enthält die Daten,
  // mit denen im Programmcode nun weitergearbeitet wird
  var name = snapshot.val();
  // in name ist nun der Kundenname gespeichert
});
```

# Datenschema NoSQL: Key-Value-Datenbanken (Firebase)

Aufgabe 2: *Zu einem Kunden ist nur die Email-Adresse bekannt. Wie gelangt man an seine Kundennummer?*

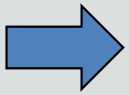
SQL:

```
SELECT KNr FROM Kunde WHERE
email='hans.mueller@mni.thm.de';
```

Firebase:

```
var ref = firebase.database().ref("Kunde");

ref.orderByChild("email").equalTo("hans.mueller@mni.thm.
de").once('value', function(snapshot) {
    var KNr = snapshot.val().KNr;
    // in KNr ist nun die Kundennummer gespeichert
});
```



Aber: Firebase unterstützt keine mehrfachen WHERE-Bedingungen!

# Datenschema NoSQL: Key-Value-Datenbanken (Firebase)

Weiterführendes Beispiel: *Es sollen alle Kunden ausgegeben werden, die mit Namen „Müller“ und Vornamen „Xaverius“ heißen.*

SQL:

```
SELECT KNr FROM Kunde WHERE
Name='Müller' AND Vorname='Xaverius';
```

Mit Firebase ist dies nicht ohne weiteres möglich!

1. Möglichkeit: Nur eine Bedingung verwenden und Rest lokal filtern.

```
var ref = firebase.database().ref(„Kunde“);

ref.orderByChild(„Vorname“).equalTo(“Xaverius”).once('va
lue', function(snapshot) {
    // im snapshot sind alle Personen mit dem
    // Vornamen Xaverius gespeichert. Nun muss für jede
    // Person überprüft werden, ob Name == „Müller“.
});
```

# Datenschema NoSQL: Key-Value-Datenbanken (Firebase)

Weiterführendes Beispiel: *Es sollen alle Kunden ausgegeben werden, die mit Namen „Müller“ und Vornamen „Xaverius“ heißen.*

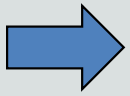
SQL:

```
SELECT KNr FROM Kunde WHERE
Name='Müller' AND Vorname='Xaverius';
```

Mit Firebase ist dies nicht ohne weiteres möglich!

2. Möglichkeit: Für jeden Kunden im `Kunde`-Pfad ein neues Attribut einfügen, das sowohl den Vornamen als auch den Nachnamen enthält.

```
Kunde
  $KNr
     Name: <String>
     Vorname: <String>
     Name_Vorname: <String>
     Email: <String>
```



Beispiel: “Müller_Xaverius”

# Datenschema NoSQL: Key-Value-Datenbanken (Firebase)

Weiterführendes Beispiel: *Es sollen alle Kunden ausgegeben werden, die mit Namen „Müller“ und Vornamen „Xaverius“ heißen.*

SQL:

```
SELECT KNr FROM Kunde WHERE
Name='Müller' AND Vorname='Xaverius';
```

Mit Firebase ist dies nicht ohne weiteres möglich!

2. Möglichkeit: Für jeden Kunden im `Kunde`-Pfad ein neues Attribut einfügen, das sowohl den Vornamen als auch den Nachnamen enthält.

```
var ref = firebase.database().ref(„Kunde“);

ref.orderByChild(„Name_Vorname“).equalTo(“Müller_Xaveriu
s”).once('value', function(snapshot) {
    // Im snapshot sind alle Kunden mit dem
    // Namen „Müller“ und dem Vornamen „Xaverius“
    // enthalten.
});
```

# Datenschema NoSQL: Key-Value-Datenbanken (Firebase)

Aufgabe 3: *Für die Produkte soll es im Online-Shop eine Listenansicht geben, wo nur wenige Produktinformationen (PNr, Bild, Bezeichnung und Preis) angezeigt werden sollen.*

SQL:

```
SELECT PNr, BildUrl, Bezeichnung, Preis
FROM Produkt
```

Firebase:

1. Möglichkeit:
Alle in der Listenansicht anzuzeigenden Produkte werden vom Pfad `Produkt/` ausgelesen. Problem: Das ausgelesene Objekt (Snapshot) enthält aufgrund der Black-Box-Problematik alle Attribute. → Hoher Traffic-Verbrauch!

2. Möglichkeit:
Einen eigenen Pfad anlegen für die Listenansicht der Produkte:

```
ProduktListenansicht
   $PNr
      BildUrl: <String>
      Bezeichnung: <String>
      Preis: <Double>
```

# Datenmanipulation
# NoSQL: Key-Value-Datenbanken (Firebase)

Aufgabe 3: *Für die Produkte soll es im Online-Shop eine Listenansicht geben, wo nur wenige Produktinformationen (PNr, Bild, Bezeichnung und Preis) angezeigt werden sollen.*

SQL:

```
SELECT PNr, BildUrl, Bezeichnung, Preis
FROM Produkt
```

Firebase:

1. Möglichkeit:
Alle in der Listenansicht anzuzeigenden Produkte werden vom Pfad `Produkt/` ausgelesen. Problem: Das ausgelesene Objekt (Snapshot) enthält aufgrund der Black-Box-Problematik alle Attribute. → Hoher Traffic-Verbrauch!

2. Möglichkeit:
Einen eigenen Pfad anlegen für die Listenansicht der Produkte:

```
var ref = firebase.database().ref('ProduktListenansicht');

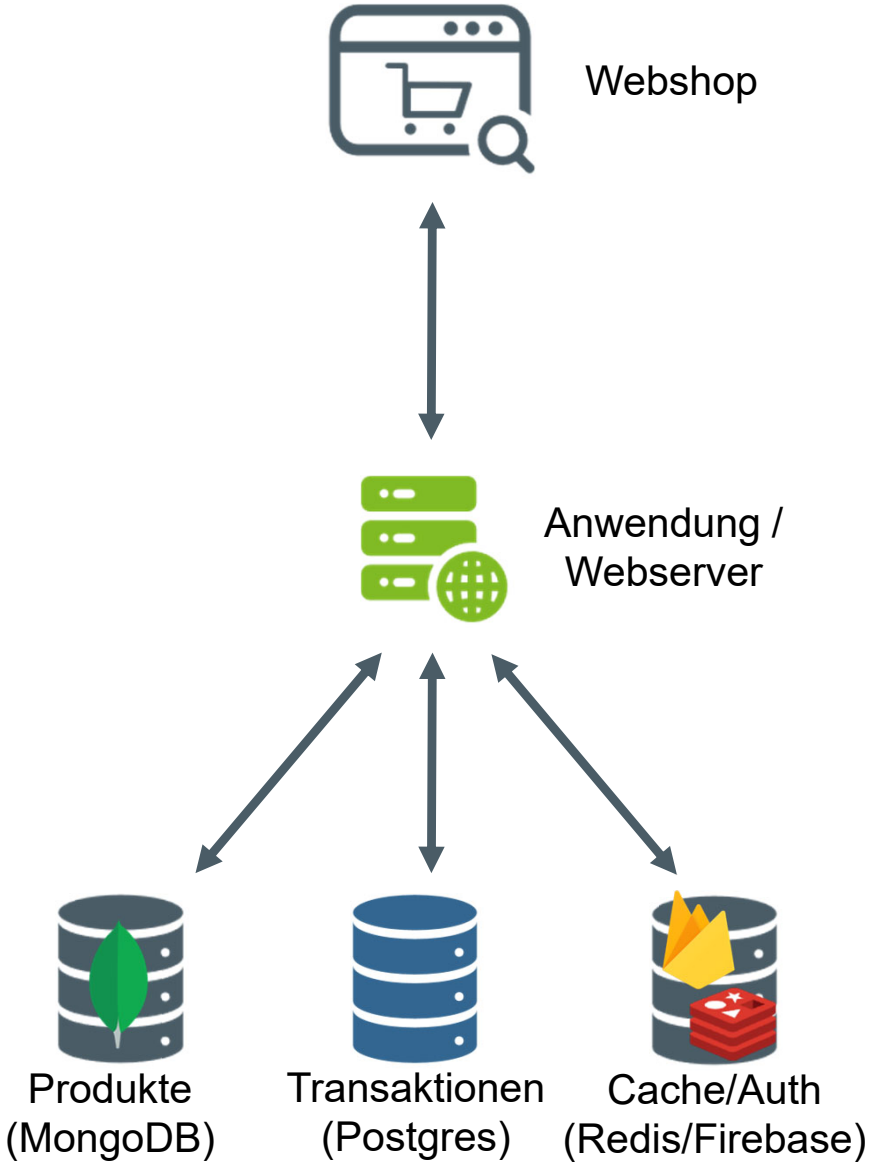
ref.once('value', function(snapshot) {
   ... // Im snapshot sind alle Values des Pfades enthalten
});
```

# 10 Der Onlineshop

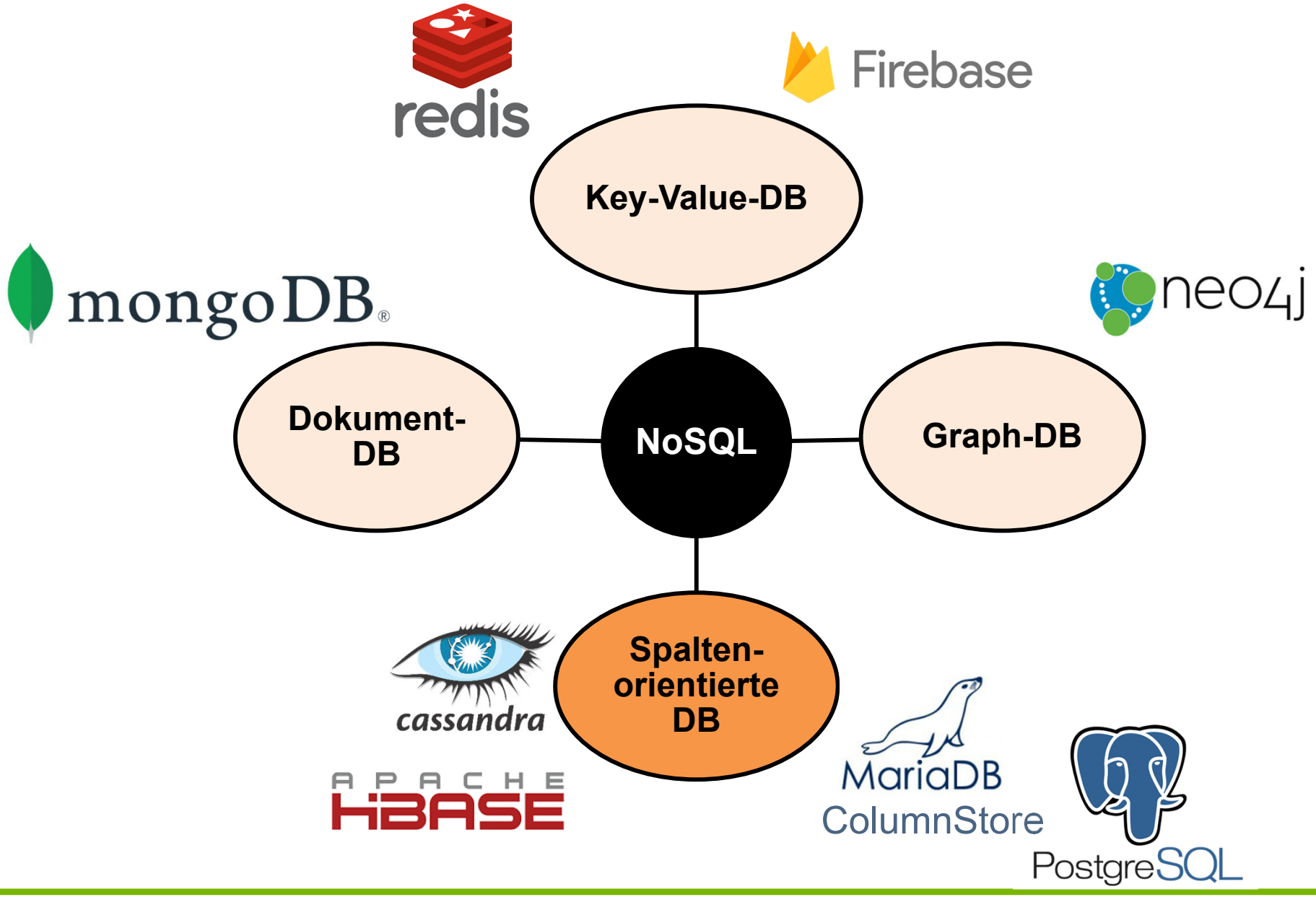
- Verkauft **diverse Artikel**
- Startseite:
  - Neue Artikel
  - **Meistverkaufte Artikel** ✓
- Daten:
  - Kunden
  - Bestellungen
  - Produkte
- **Authentifikation** ✓

Neue Anforderung:

- Data-Mining auf Kundendaten

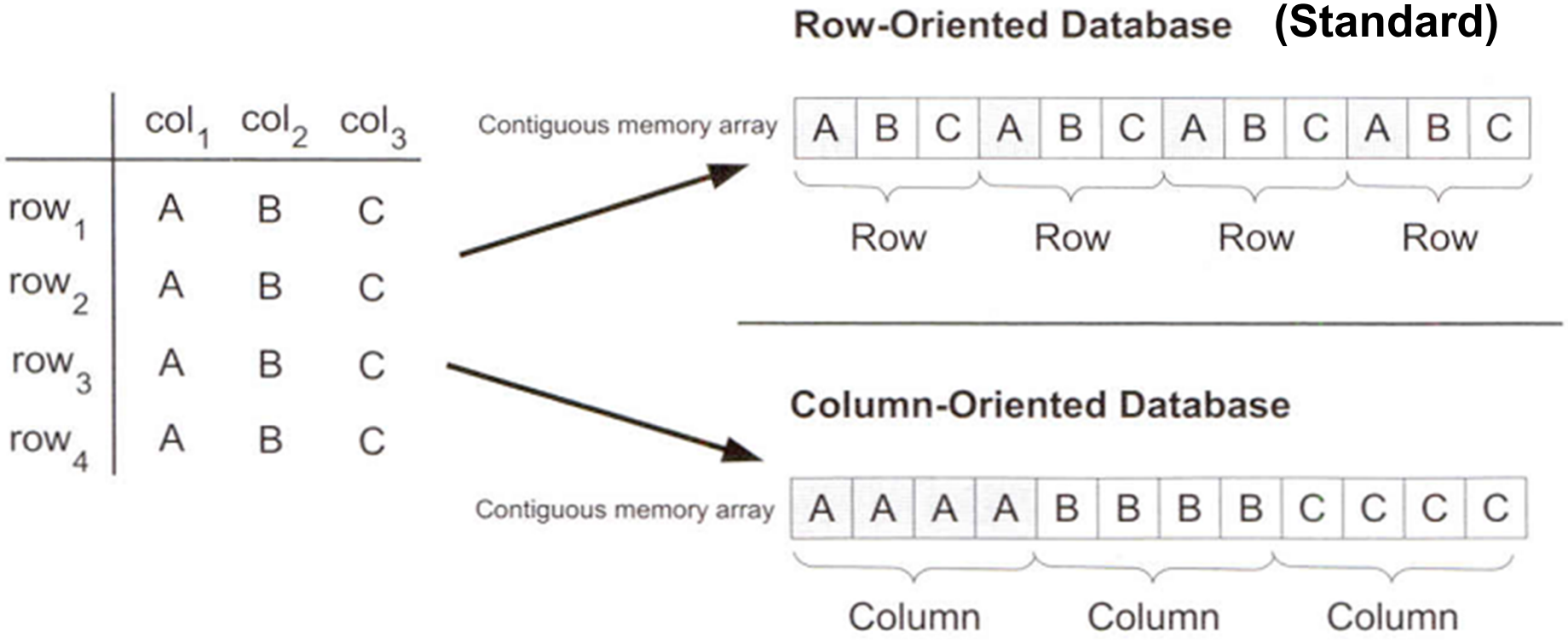


# 10 Kerntechnologien im Bereich NoSQL



# 10 Exkurs: Spaltenorientierte Datenbanken

- Wie bei den zeilenorientierten kann bei einigen spalten-orientierten Datenbanken **SQL** zum Einsatz kommen.
- **Speicherung der Daten** erfolgt in einer anderen Form: Spaltenorientiert: Werte einer Spalte stehen zusammen



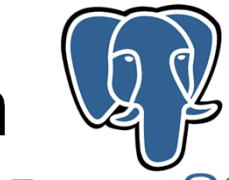
# 10 Spaltenorientierte Datenbanken

- Eine SQL-Datenbank stellt ihre Daten als zwei-dimensionale Tabellen aus Zeilen und Spalten dar
- Diese müssen aber in eindimensionaler Form gespeichert werden.

| PNr | Bezeichnung | Preis | BildUrl |
|---|---|---|---|
| 1 | Data Science for Business | 20,99 | 51zr9I3Kn.jpg |
| 2 | Data Analytics | 19,95 | 81z544Kn.jpg |
| 3 | Data Science Essentials in Python | 29,99 | 207f664Kn.jpg |

1, Data Science for Business, 20.99, 51zr9I3Kn.jpg;
2, Data Analytics, 19.95, 81z544Kn.jpg;
3, Data Science Essentials in Python, 29.99, 207f664Kn.jpg;

# Spaltenorientierte Datenbanken










- Eine spaltenorientierte Datenbank geht stattdessen Spalte für Spalte vor:

| PNr | Bezeichnung |
|---|---|
| 1 | Data Science for Business |
| 2 | Data Analytics |
| 3 | Data Science Essentials in Python |

| PNr | Preis |
|---|---|
| 2 | 19,95 |
| 3 | 29,99 |
| 1 | 20,99 |

| PNr | BildUrl |
|---|---|
| 3 | 207f664Kn.jpg |
| 1 | 51zr9I3Kn.jpg |
| 2 | 81z544Kn.jpg |

1, Data Science for Business, 20.99, 51zr9I3Kn.jpg;
2, Data Analytics, 19.95, 81z544Kn.jpg;
3, Data Science Essentials in Python, 29.99, 207f664Kn.jpg;

# 10 Spalten- vs. Zeilenorientierte Datenbanken

- Spaltenorientierte Systeme sind effizienter, wenn eine Spalte gleichzeitig für alle Zeilen der Tabelle einen neuen Wert erhält, da man die Spaltendaten effizient schreiben kann und die Daten der anderen Spalten nicht berücksichtigen müssen.
  - Beispiel: UPDATE Produkt SET Preis = Preis * 1.25;
- Spaltenorientierte Systeme sind auch dann effizienter, wenn ein Aggregat über viele Zeilen, aber nur wenige Spalten gebildet werden muss, da man dann im Gegensatz zum zeilenorientierten System nur diese und nicht alle Spalten lesen muss.
  - Beispiel: SELECT SUM(Preis) FROM Warenkorb;

# 10 Spalten- vs. Zeilenorientierte Datenbanken

- Zeilenorientierte SQL-Systeme sind effizienter, wenn gleichzeitig viele Spalten einer einzigen Zeile benötigt werden und wenn die Zeilenbreite sehr groß ist, da dann die ganze Zeile mit einem einzigen Plattenzugriff gelesen werden kann.
  - Beispiel: SELECT * FROM Kunden WHERE KNr = 101;
- Zeilenorientierte SQL-Systeme sind beim Einfügen einer neuen Zeile effizienter, wenn alle Daten dieser Zeile auf einmal vorliegen, da dann die Zeile mit einem einzigen Zugriff geschrieben werden kann.
  - Beispiel: INSERT INTO Kunden (Nachname, Vorname, Email) VALUES ('Maier', 'Karl-Heinz', 'Karl.M@mni.thm.de');

# 10 Spaltenorientierte Datenb.: Anwendungszweck

- Für **analytische Informationssysteme** bei denen die direkte Verarbeitung von Informationen benötigt ist z.B.:
  - für Data-Mining Systeme
  - Speicherung und Auswertung von Maschinendaten (IoT)
  - Schreibintensive Anwendungen

- Weitere Anwendungen:
  - Suchmaschinen wie Google (DB: BigTable)
  - riesige Data Warehouses, in denen Millionen von Video- und Sound-Dateien gespeichert sind und eine Schnellsuche benötigt wird (YouTube)
  - soziale Netzwerke (Facebook, Pinterest, Twitter).

# 10 Zeilen- vs. spaltenorientierte DB Datenmanipulation

- Veränderung der Tabellenstruktur

```
ALTER TABLE Kunde ADD COLUMN Geburtsdatum Date
```

  – ZO: Verschieben jedes Tupels (Shift) und Erneuern der Tabelle (Rebuild), enormer Aufwand je nach Datenmenge in Tabelle
  \+ SO: Einfaches Hinzufügen einer neuen leeren Spalte

- Einfügen eines neuen Datensätze (Gleiches gilt für das Aktualisieren eines kompletten Datensatzes)

```
INSERT INTO Kunde (Nachname, Vorname, ...) VALUES
('Maier', 'Karl-Heinz', ...)
```

  \+ ZO: Anhängen des neuen Tupels ans Ende der Tabelle (evtl. Index-Aktualisierung)
  – SO: Öffnen und schreiben am Ende jeder Spalte (viele Positionen)

# 10 Zeilen- vs. spaltenorientierte DB Datenmanipulation

- Aktualisieren eines Wertes

```
UPDATE Produkt SET Preis = '23,99' WHERE PNr = 16
```

  - ZO: Suchen nach Datensatz und ändern des Preises
  - SO: Suchen nach Datensatz und ändern des Preises

- Aktualisieren einer Spalte

```
UPDATE Produkt SET Preis = Preis * 1,25
```

  – ZO: Suchen und Ändern der Spalte in jedem Tupel
  \+ SO: Aktualisierung in nur einer Spalte (Zusammenhängender Speicherbereich)

# 10 Zeilen- vs. spaltenorientierte DB Datenmanipulation

- Auslesen einzelner kompletter Datensätze

```
SELECT * FROM Kunde WHERE KNr = 101
```

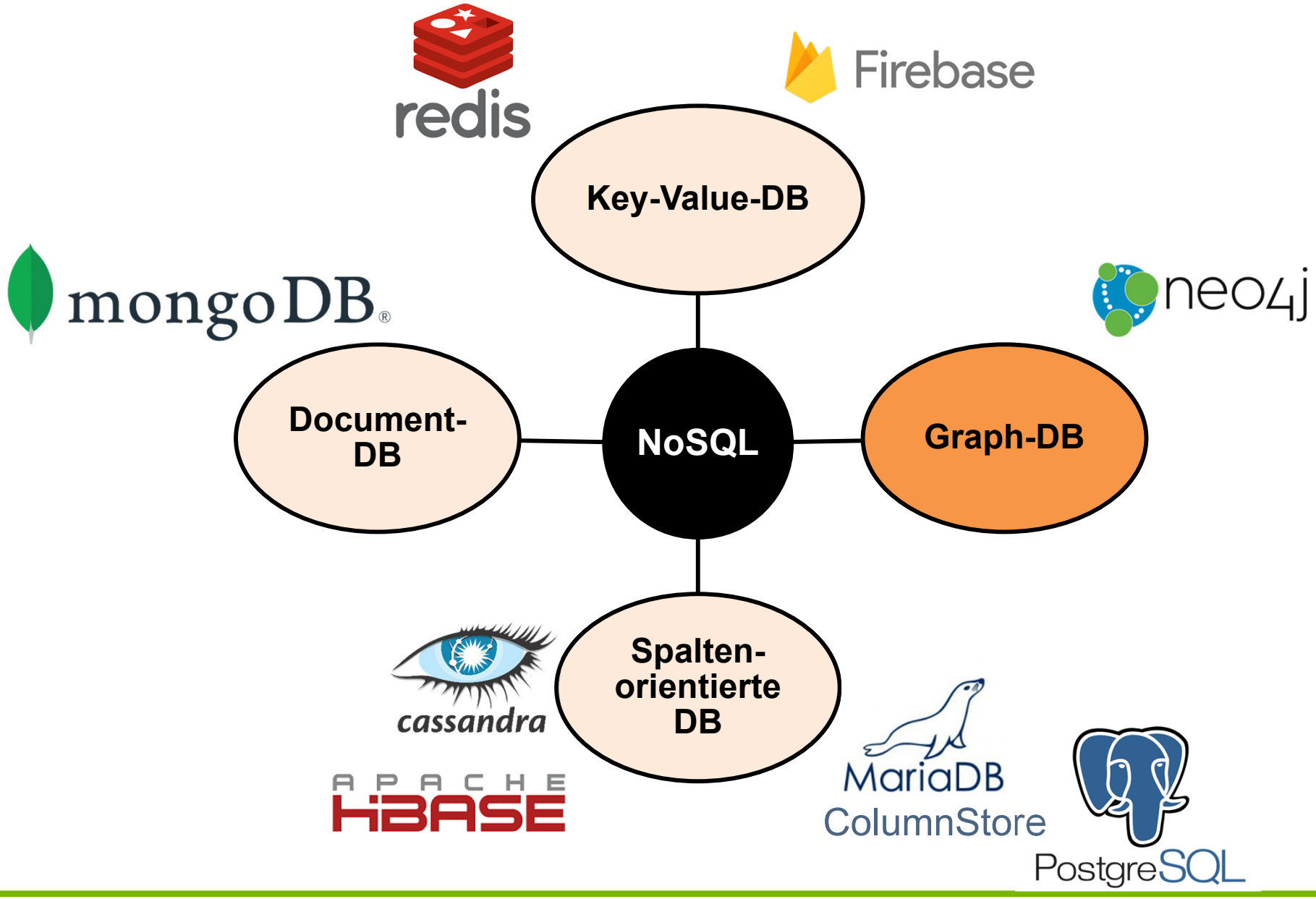
  \+ ZO: Auslesen des gesamten Tupels mit einem Zugriff (Performance-Vorteile mit Index),
  ABER: Scan über gesamte Tabelle, wenn Index nicht vorhanden
  SO: Suche in allen Spalten nach den Daten der KNr = KNr

- Aggregation / Auslesen vieler Datensätze (unter Verwendung weniger Spalten)

```
SELECT SUM(Preis) FROM Warenkorb
```

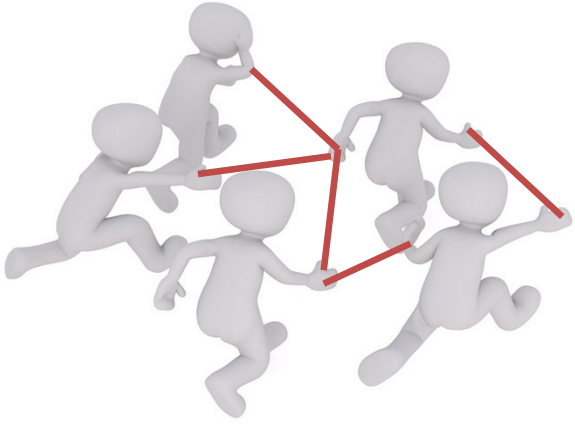
  – ZO: Lesen gesamter Tabelle
  \+ SO: Nur Lesen der Daten für relevante Spalte(n)

# 10 Kerntechnologien im Bereich NoSQL



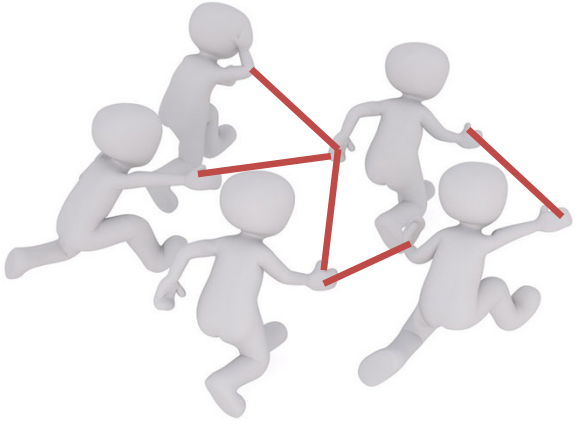
# 10 Grundlagen NoSQL: Graph-Datenbanken

- Basiert auf der **Graphentheorie**
- Eignet sich für stark vernetzte Informationen
- **Beziehungen** stehen im Vordergrund
  - Neue Beziehungen lassen sich leicht anlegen und ändern
  - Informationen hängen stark zusammen: Horizontale Skalierung schwierig
- **Traversierung** des Graphen
- Suchalgorithmen (Tiefen-/Breitensuche, der Graphentheorie werden angewandt
  - Kürzeste Pfade
  - Cliquen (Agglomerate im Netzwerk)
  - ...



# 10 Grundlagen NoSQL: Graph-Datenbanken

- Sie werden in Form von Graphen mit **Nodes (Knoten)** für die Entitäten und gerichteten **Edges (Kanten)** für die Beziehungen zwischen ihnen modelliert
- Jeder **Knoten** bekommt eine eigene, einzigartige Bezeichnung und enthält zudem **Eigenschaften in Form von Key-Value-Paaren**



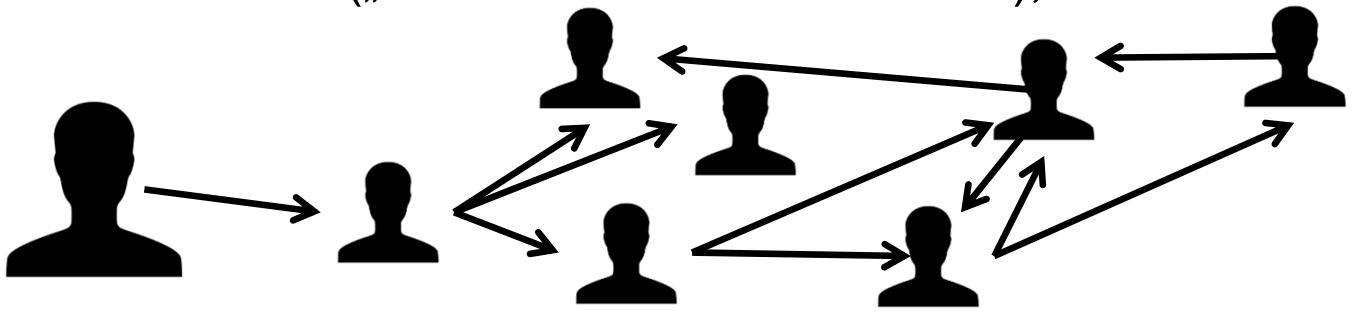
# Grundlagen NoSQL: Graph-Datenbanken

- Auch die **Kanten** haben eine eindeutige Bezeichnung, Eigenschaften und außerdem Informationen zu ihrem Start- und Endknoten
- Ein **Schema** muss im Vorhinein nicht definiert werden, Knoten, Eigenschaften, Verbindungen und neue Beziehungsarten können jederzeit dynamisch ergänzt oder entfernt werden
- Wenn es um **komplexe Beziehungen innerhalb großer Datenmengen** geht, sind Graphdatenbanken klar im Vorteil gegenüber den relationalen DBMS, aber auch den anderen NoSQL-Datenbanken

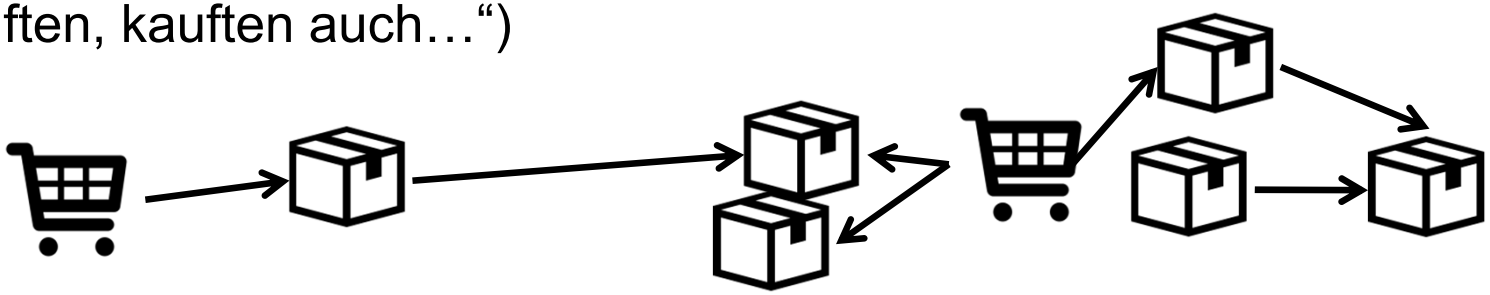
# 10 Anwendungsgebiete

Graphdatenbanken eignen sich besonders für solche Szenarien, in denen vernetzte Beziehungen in den Daten vorliegen z.B.:

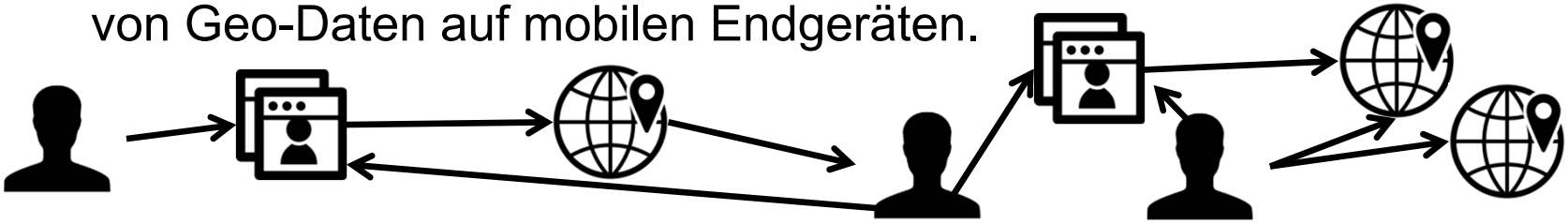
- Soziale Netzwerke („Wer kennt wen über wen?“),



- Kaufempfehlungen in Onlineshops („Kunden, die dieses Produkt kauften, kauften auch…“)



- Das Bereitstellen personalisierte Werbeinhalte unter Einbeziehung von Geo-Daten auf mobilen Endgeräten.



# 10 Neo4j - Beispiel

```
CREATE (pos1:Artikel { name: "Datenbanksysteme - Eine
Einführung", autor: "Alfons Kemper", preis: 19 })
```

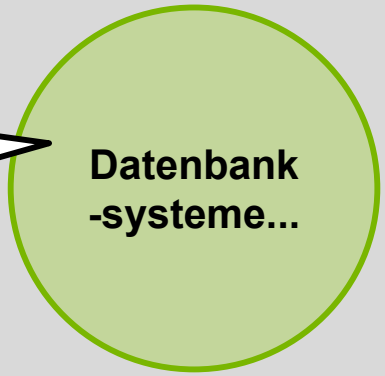
- `CREATE` zum Erstellen von sogenannten Nodes mit Daten
- `pos1:Artikel` Eine Variable `pos1` und Label `Artikel` für den neuen Node
- `{ }` Klammern, um dem Node beliebige Eigenschaften hinzuzufügen

Denkbar wäre z.B. zusätzlich:

```
{…Verlag: "De Gruyter",
  Auflage: 10,
  Seiten: 880,
  Sprache: "Deutsch",
  ISBN: 3100443759…}
```

# 10 Neo4j - Beispiel

```
MATCH (pos1:Artikel) WHERE pos1.name =
"Datenbanksysteme - Eine Einführung"
RETURN pos1;
```




- `MATCH`-Klausel, um ein Muster von Nodes und Beziehungen anzugeben
- `(pos1:Artikel)` ein einziges dieser Muster mit der Variable `pos1` und dem Label `Artikel`
- `WHERE`-Klausel zur Restriktion der Ergebnisse
- `pos1.name = "Datenbanksysteme…"` vergleicht die name-Eigenschaft mit dem Wert `"Datenbanksysteme…"`
- `RETURN`-Klausel um Ergebnisse (Attribute oder ganzes Objekt) anzufordern

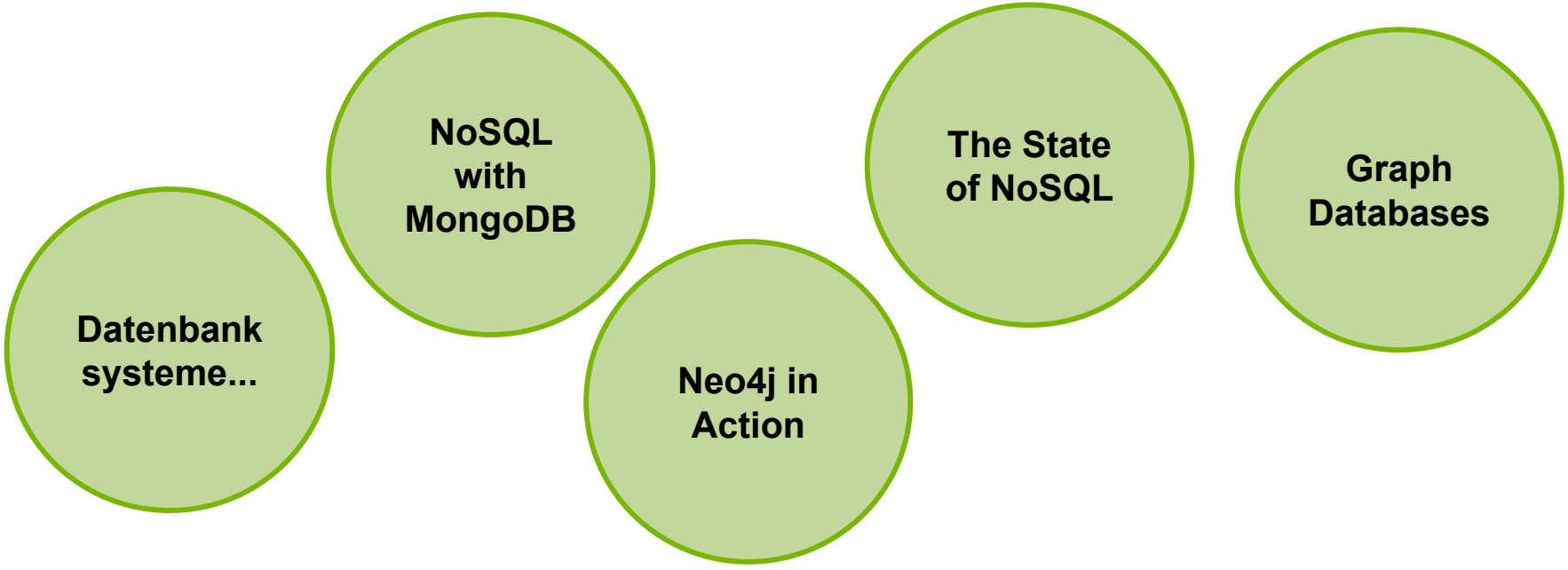
# 10 Neo4j - Beispiel

```
CREATE (pos2:Artikel { name: "Neo4j in Action", from:
"Ted Hills", preis: 31 })
```



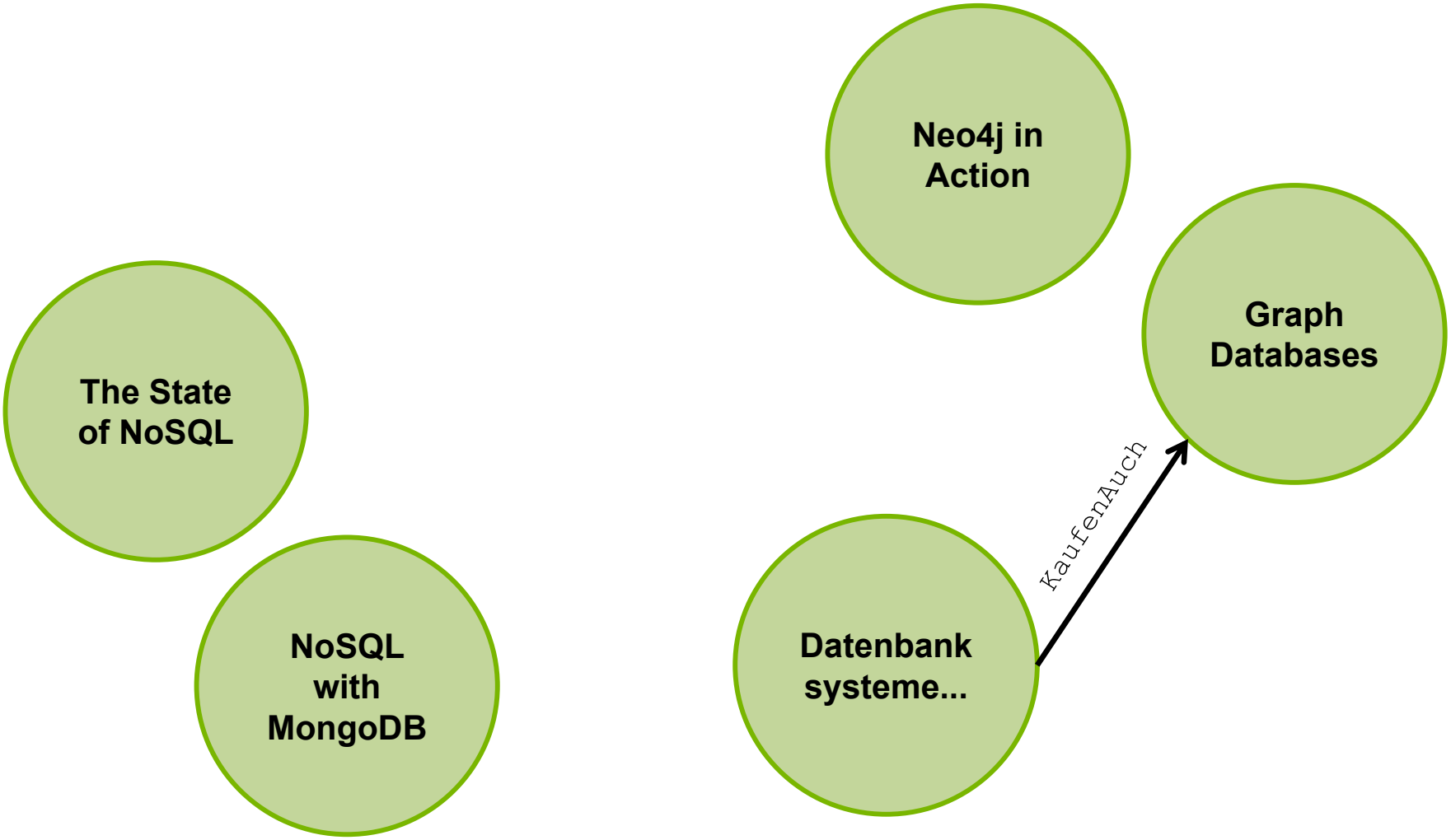
# 10 Neo4j - Beispiel

```
CREATE
(pos3:Artikel { name: "The State of NoSQL 2016", from:
"Adam Fowler", preis: 2 }),
(pos4:Artikel { name: "NoSQL with MongoDB in 24 Hours",
from: "Brad Dayley", preis: 23 }),
(pos5:Artikel { name: "Graph Databases", from: "Ian
Robinson", preis: 25 })
```



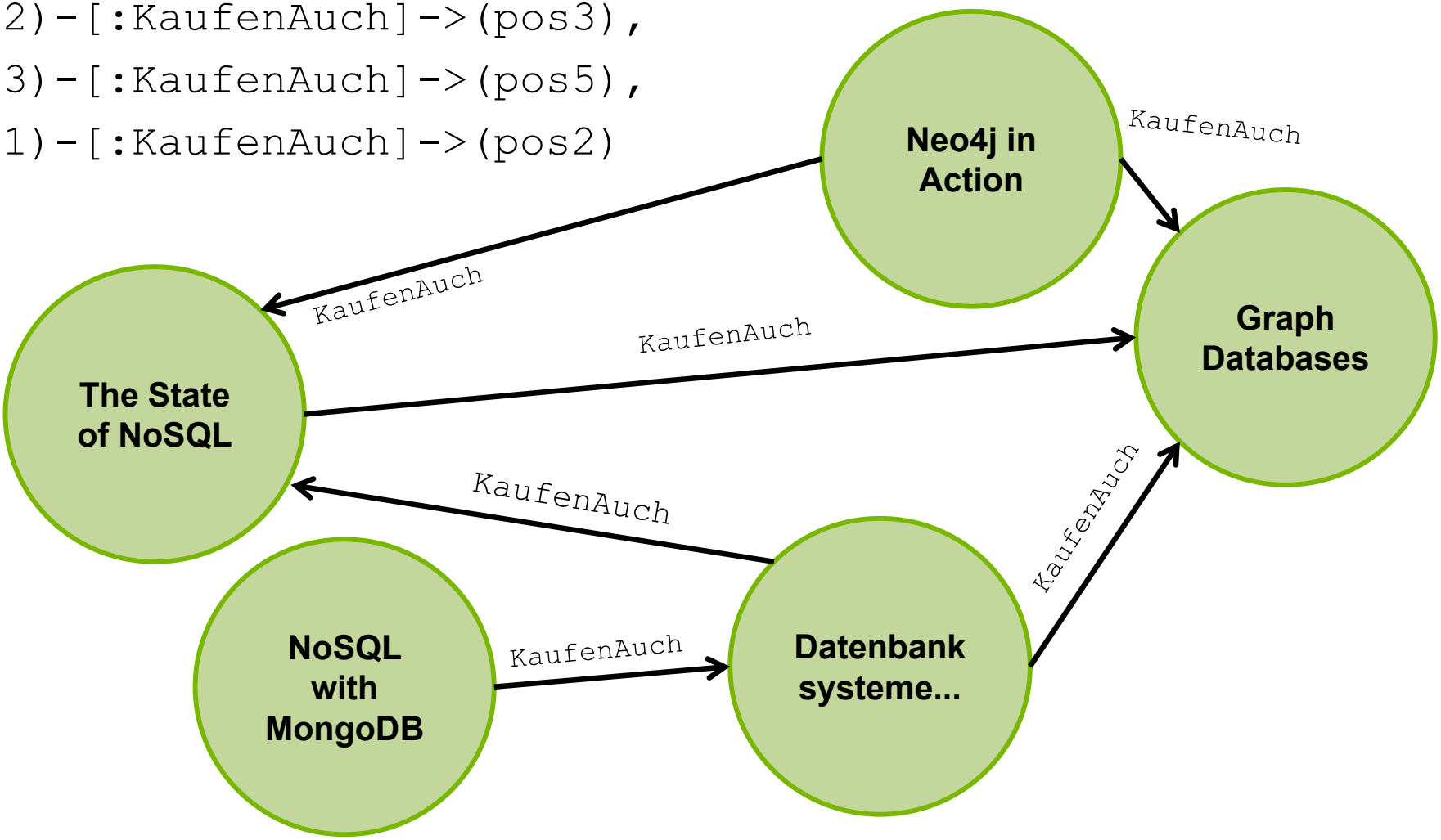
# 10 Neo4j - Beispiel

```
(pos1)-[:KaufenAuch]->(pos5)
```



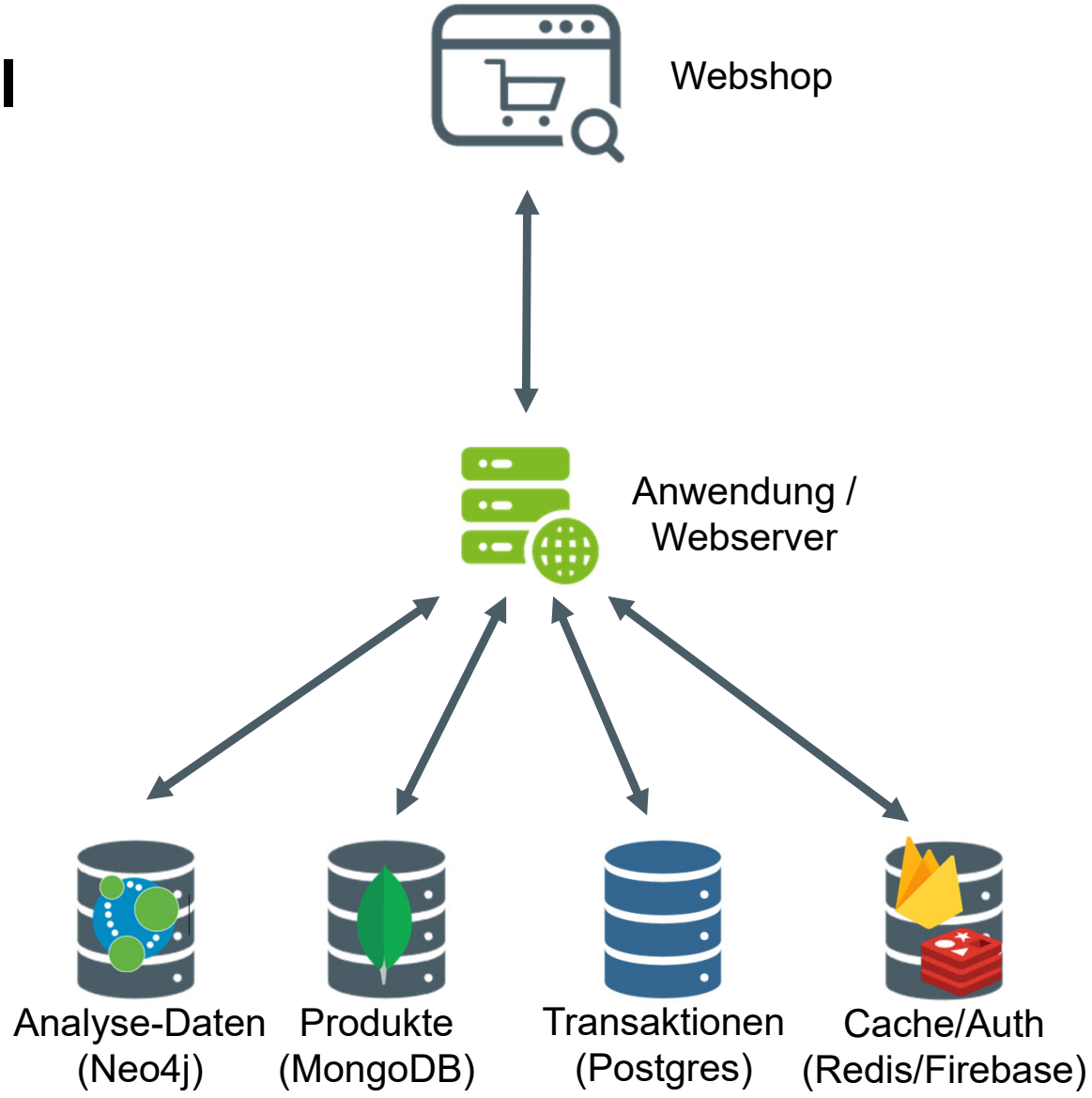
# 10 Neo4j - Beispiel

```
(pos4)-[:KaufenAuch]->(pos1),
(pos2)-[:KaufenAuch]->(pos5),
(pos2)-[:KaufenAuch]->(pos3),
(pos3)-[:KaufenAuch]->(pos5),
(pos1)-[:KaufenAuch]->(pos2)
```



# 10 Der Onlineshop

- Verkauft **diverse Artikel**
- Startseite:
  - Neue Artikel
  - **Meistverkaufte Artikel**
- Daten:
  - Kunden
  - Bestellungen
  - Produkte
- **Authentifikation**
- **Data-Mining** ✓
- Neue Anforderung: …



# 10 Beispielhaftes Wachstum eines Webshops

**Merke:**

Das KISS Prinzip gilt auch für Datenbanken!

**K**eep
**I**t
**S**imple,
**S**tupid
















Datenanalyse

Caching von komplexen Abfragen



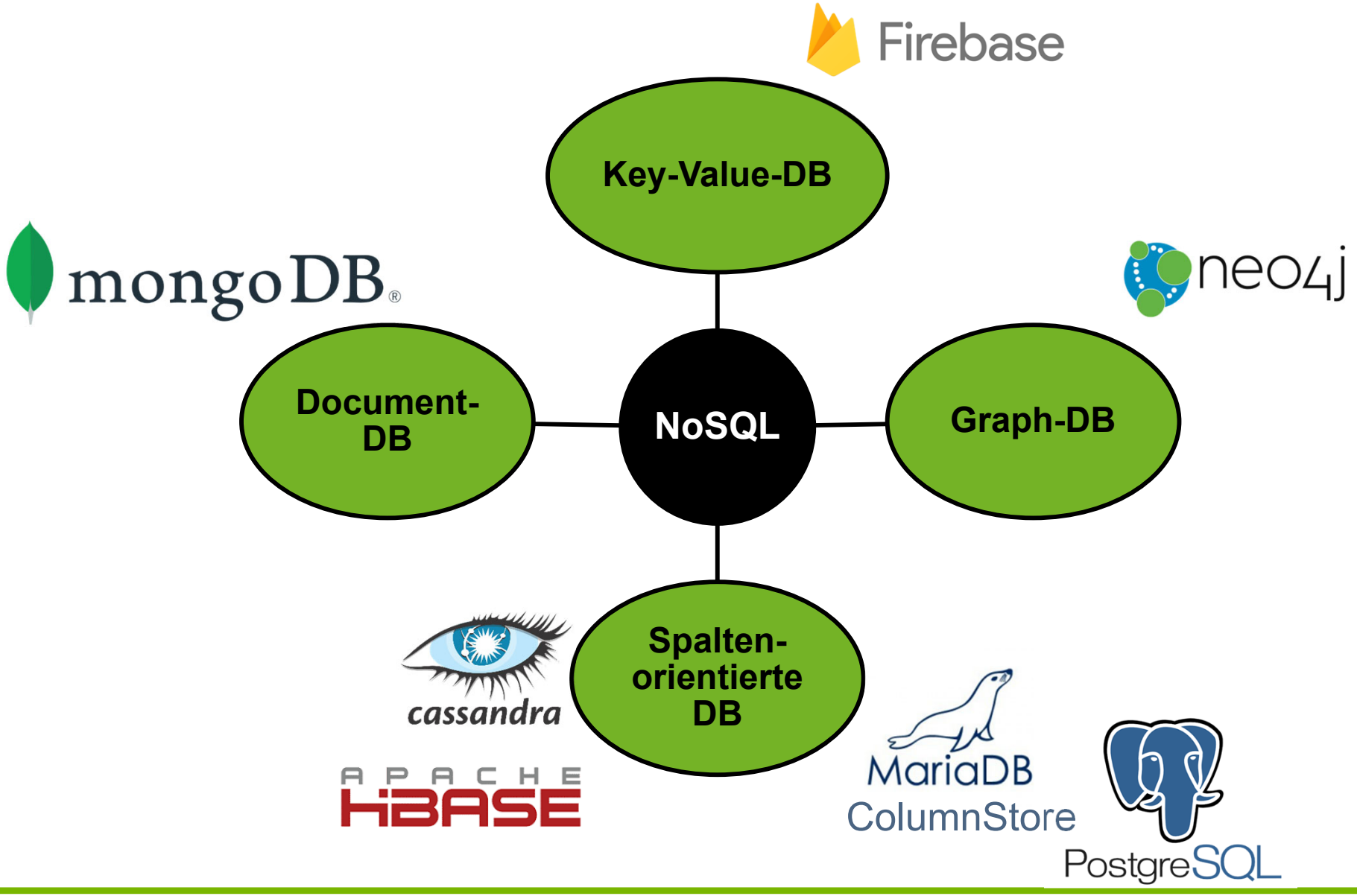


Diversere Produktpalette



Kleiner, spezifischer Webshop

# Kerntechnologien im Bereich NoSQL






## (1) Alternative 1: **Installieren Sie sich eine MongoDB lokal**

- **Docker:** Installieren Sie Docker (https://docs.docker.com/desktop/ oder https://docs. docker.com/engine/install/) und führen Sie danach in der Kommandozeile als einen Befehl folgendes aus:

```
docker run -d
-e MONGO_INITDB_ROOT_USERNAME=admin
-e  MONGO_INITDB_ROOT_PASSWORD=Ihr-sicheres-Passwort
-p 127.0.0.1:27017:27017 --name mongodb mongo
```

- Ändern Sie dabei das Passwort. Sie können sich im nächsten Schritt nun in Compass mit der folgenden URL verbinden:

```
mongodb://admin:Ihr-sicheres-Passwort@localhost:27017/
```

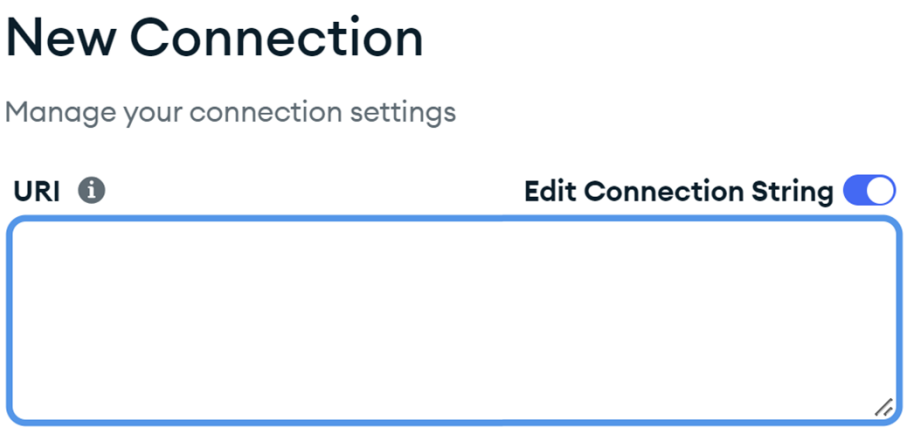


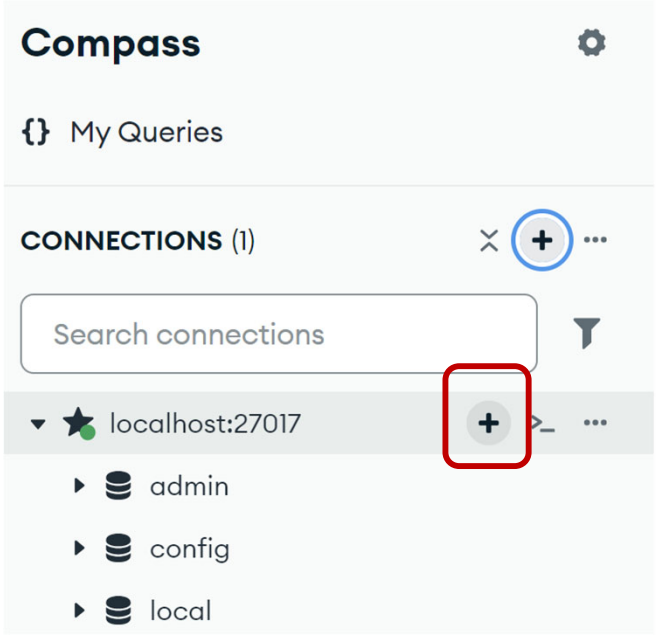

## (1) Alternative 2: **Nutzen Sie eine MongoDB aus der Cloud**

- Registrieren Sie sich unter https://cloud.mongodb.com/, nachdem Sie Ihre E-Mail- Adresse bestätigt haben, können Sie auf 'Create' klicken und Informationen zu Ihrer Datenbank wie den Namen angeben. Achten Sie dabei darauf, dass Sie M0 (kostenlos) auswählen.
- Im darauffolgenden Dialog können Sie einstellen, von welcher IP-Adresse aus auf Ihre Datenbank zugegriffen werden kann. Dort wählen Sie am besten 'alle IPs freigeben' (empfohlen, da wir nur experimentieren und keine sensiblen Daten speichern).
- Setzen Sie danach einen Nutzernamen und Passwort, wählen Sie die Verbindungsart 'Compass' und 'Ich habe Compass bereits installiert'. Dann kopieren Sie die Verbindungs-URL, welchen wir dann nach der Installation von Compass nutzen.

## (2) Installieren Sie MongoDB Compass

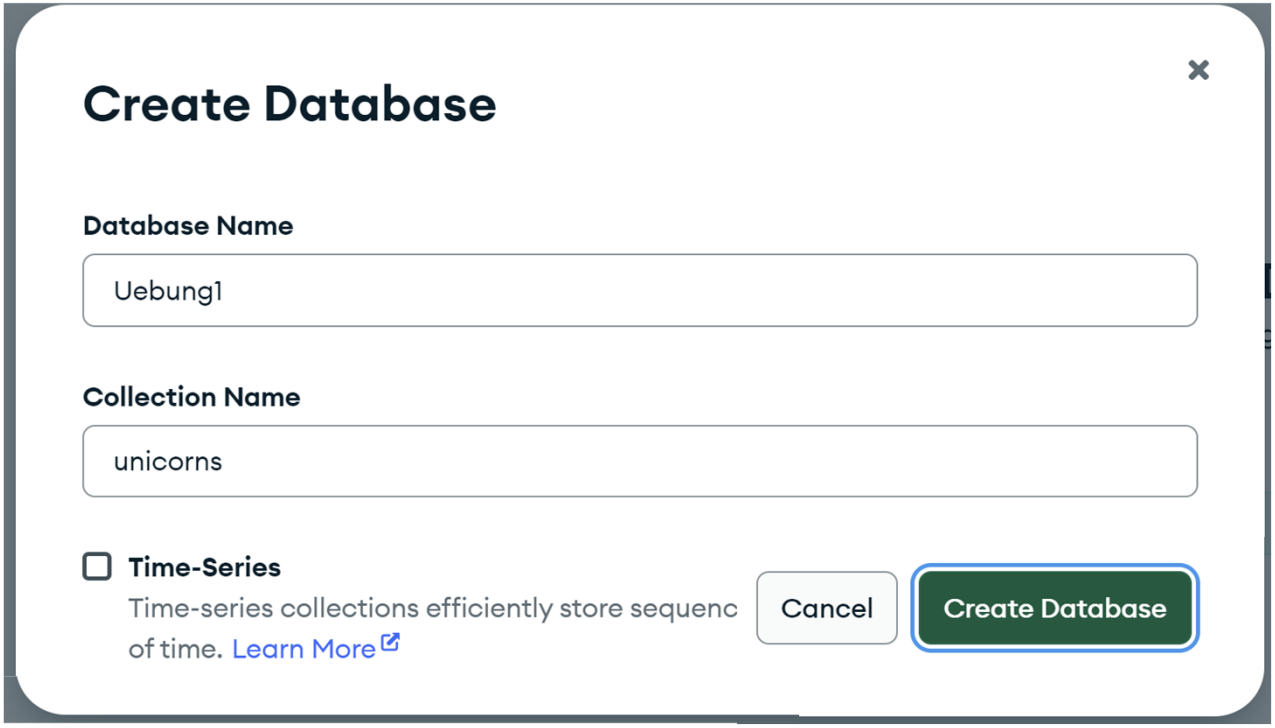
- Installieren Sie MongoDB Compass von folgender URL:
  https://www.mongodb.com/try/download/compass
- Nach dem Öffnen des Programms die URL von zuvor eingeben.




- Dann eine neue Datenbank anlegen. ➔ ➔ ➔
  DB-Name: Uebung1
  Collection-Name: unicorns

# 10 Aufgaben




und zuletzt oben rechts drücken: 


(3) Erstellen Sie nun ein erstes weibliches Wesen mit Name **Aurora** und Gewicht **450** in der Collection **unicorns**.

(4) Erstellen Sie eine erste Abfrage zur Überprüfung des erstellten Dokuments in der Collection **unicorns**.

Erwartet Antwort (4): { "_id" : ObjectId("…"), "name" : "Aurora", "gender": "f", "weight" : 450 }




(5) Fügen Sie nun zwei weitere Einhörner ein und variieren Sie dabei auch die Anzahl und Typen der verschiedenen Felder: Ein Wesen hat Essensvorlieben, bei einem anderen geben Sie sein Geburtsdatum an.

(6) Lassen Sie sich nun nochmals alle bis jetzt eingefügte Einhörner anzeigen.

(7) Löschen Sie alle erstellten Einhörner und überprüfen dann, dass die Collection **unicorns** leer ist.

# 10 Aufgaben




(8) Laden Sie sich von Moodle die Datei **mongoDB_beispieldaten.txt** herunter und spielen die Beispieldaten hieraus in Ihre mongoDB ein. Lassen Sie die DB die Anzahl an Dokumenten zählen.

(9) Lassen Sie sich nun die Einhörner mit den folgenden Eigenschaften anzeigen:

a) mit Namen Aurora
b) alle Weibchen, welche Äpfel mögen (Begriffe bitte passend übersetzen)
c) alle Weibchen, welche Äpfel und Karotte mögen
d) alle Weibchen, welche Äpfel oder Karotte mögen
e) noch keine Vampire gesehen (auch keinen Eintrag **vampires** oder =0)
f) alle Männchen, welche ihr Gewicht zwischen 600 und 900 haben

Hilfe zu den Query Operatoren finden Sie unter:
https://www.mongodb.com/docs/manual/reference/operator/query/

# 10 Kontrollfragen

- Was sind Key-Value-Datenbanken? Was ist der Unterschied zu dokumentenbasierte Datenbanken?
- Was sind spaltenorientierte Datenbanken? Welche Anwendungen haben sie?
- Wo kommen Graph-Datenbanken zum Einsatz?
- Warum werden relationalen Datenbanken gebraucht? Warum sind NoSQL-Datenbanken nützlich?